

GNSS SPOOFING/JAMMING DETECTOR USING RISC V

PROJECT REPORT

submitted by

NIKHIL VINOD (MDL22EC089)

PRANAV S (MDL22EC097)

VAISHAKH MOHAN (MDL22EC121)

VISHNU S NAMBIAR (MDL22EC124)

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology in

Electronics and Communication Engineering



Department of Electronics Engineering

Model Engineering College

Thrikkakara

April 2026

Vision and Mission of the Institute and Department

Vision and Mission of the Institute	Vision and Mission of the Department
Vision	Vision
Evolve into an academy of excellence to serve the knowledge society	Mould professionally competent and socially committed electronics engineers capable of innovations and entrepreneurial initiatives for addressing the changing needs of the world
Mission	Mission
M₁ Implement quality education through teaching-learning process M₂ Inculcate culture of technical innovations and creativity M₃ Instil high standards of professional ethics and values	M₁ Instil quality of technical expertise and scholarly research in our stakeholders M₂ Inculcate ethical standards and leaderships qualities in our stakeholders through various activities M₃ Equip the students to apply technology for the benefit of the society

Course Objectives

- To apply engineering knowledge in practical problem solving
- To foster innovation in design of products, processes or systems.
- To develop creative thinking in finding viable solutions to engineering problems.

Course Outcomes

Course Outcomes	
CO ₁	Model and solve real world problems by applying knowledge across domains.
CO ₂	Develop products, processes or technologies for sustainable and socially relevant applications.
CO ₃	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.
CO ₄	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.
CO ₅	Identify technology/research gaps and propose innovative/creative solutions.
CO ₆	Organize and communicate technical and scientific findings effectively in written and oral forms



Department of Electronics Engineering

Model Engineering College

Thrikkakara

BONAFIDE CERTIFICATE

This is to certify that this Project report entitled

GNSS SPOOFING/JAMMING DETECTOR USING RISC V

Submitted by:

NIKHIL VINOD (MDL22EC089), PRANAV S (MDL22EC097),

VAISHAKH MOHAN (MDL22EC121), VISHNU S NAMBIAR (MDL22EC124)

is a bonafide account of the work done by them under our supervision

Ms. Aparna Devi P S

Project Guide

Ms. Jibi John

Project Coordinator

Dr. Jobymol Jacob

Head of the Department

ACKNOWLEDGEMENT

First and foremost, we would like to express our heartfelt gratitude to the Lord Almighty for showering his blessings and giving us the strength and determination to successfully complete this project.

*We express our sincere thanks to **Prof. Dr. Jayachandran E. S**, Principal, Govt. Model Engineering College, Thrikkakara, for providing us with the opportunity and environment to carry out this project work at the college.*

*We are deeply thankful to **Prof. Dr. Jobymol Jacob**, Professor and Head of the Department of Electronics Engineering, for her encouragement, guidance, and constant support throughout the course of this project.*

*We extend our gratitude to our project coordinator, **Ms. Jibi John**, Associate Professor, Department of Electronics Engineering, for her valuable suggestions, motivation, and continuous assistance during the project.*

*We owe our deepest appreciation to our project guide, **Ms. Aparna Devi P S**, Associate Professor, Department of Electronics Engineering, for her expert guidance, constructive feedback, and continuous encouragement that enabled us to complete this project successfully.*

We would also like to thank all the faculty and non-teaching staff members of the Department of Electronics Engineering for their kind cooperation and help.

Finally, we are grateful to our parents and friends for their unwavering support, patience, and encouragement throughout the completion of this project.

ABSTRACT

This project presents the design and implementation of a lightweight, real-time detector for Global Navigation Satellite System (GNSS) spoofing and jamming attacks. Given the critical vulnerability of GNSS to signal interference, this system provides a robust solution by analyzing key signal integrity metrics extracted from a GNSS receiver, such as average Signal-to-Noise Ratio (SNR) and pseudorange standard deviation. The core of the detector is a Support Vector Machine (SVM) classifier, which has been trained on a comprehensive, labeled dataset to accurately distinguish between "Normal," "Jammed," and "Spoofed" signal states. This trained model is designed for implementation on a RISC-V softcore processor synthesized within a Field-Programmable Gate Array (FPGA), enabling an efficient, low-power, and hardware-based classification system. The final system is intended to monitor live GNSS data and provide immediate alerts upon the detection of an attack, enhancing the security and reliability of GNSS-dependent applications.

CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF ABBREVIATIONS	vi
LIST OF FIGURES	viii
LIST OF TABLES	x
INTRODUCTION	1
1.1 Motivation	1
1.2 Objective	2
1.3 Scope and Relevance	2
1.4 Novelty	3
1.5 Application Area	3
1.6 Organization of Thesis	3
LITERATURE REVIEW	5
2.1 Early Developments in GNSS Security and Spoofing Awareness	5
2.2 Emergence of Machine Learning-Based Detection	6
2.3 Dataset-Driven Approaches and Practical Experimentation	7
2.4 Deep Learning Advances and Their Limitations	8
2.5 Hardware-Oriented Developments and FPGA-Based GNSS Systems	9
2.6 Summary and Research Gap	10
DESIGN OF GNSS SPOOFING AND JAMMING DETECTION SYSTEM	11
3.1 System Overview	11
3.2 System Block Diagram	12
3.3 Detailed Block Description and Specifications	13
3.4 Algorithm	14
3.5 Feature Extraction and Dataset Structuring	16

3.6 SVM Classifier Design -----	17
3.6.1 Why SVM? -----	18
3.7 RISC-V Soft-Core Processor Architecture -----	19
3.8 FPGA Integration and Hardware Design -----	20
3.9 Feasibility Study -----	21
3.10 Dataset Details and Availability -----	21
3.11 Hardware Modules and Software Tools Required -----	24
3.12 Connection Diagram -----	25
3.13 Cost Analysis -----	26

IMPLEMENTATION OF GNSS SPOOFING AND JAMMING DETECTION SYSTEM -----27

4.1 Hardware Implementation Overview -----	27
4.2 Software Environment and Toolchain -----	28
4.3 Dataset Preparation and Balancing -----	29
4.4 Model Training and Hardware Conversion -----	29
4.5 System Integration and Execution -----	30
4.6 Testing and Validation -----	31
4.6.1 Software Verification Phase -----	31
4.6.2 Hardware Verification Phase -----	31

RESULTS AND DISCUSSIONS -----32

5.1 Dataset Balancing and Feature Behavior -----	32
5.2 SVM Training Results -----	33
5.3 Firmware Verification and Real-Time Hardware Performance -----	35
5.4 Real-Time Physical Actuation -----	37
5.5 FPGA Resource Utilization -----	40
5.6 System Latency and Deterministic Performance -----	42
5.7 Comparative Analysis with Existing Literature -----	43

CONCLUSION AND FUTURE SCOPE -----46

6.1 Conclusion -----	46
6.2 Future Scope -----	47

APPENDIX A: PROJECT PLAN AND WORK DIVISION -----49

A.1 PERT Chart -----	49
A.2 Gantt Chart -----	49

A.3 Work Division -----	50
APPENDIX B: SOURCE CODE -----	52
B.1 Feature Extraction Code:-----	52
B.2 SVM Training Code:-----	54
B.3 Bare-Metal Inference Engine Code: -----	56
B.4 Real-Time Data Streaming and AXI Control (Jupyter Notebook):-----	58
APPENDIX C: PYNQ-Z2 BOARD LAYOUT AND PINOUT-----	61
REFERENCES -----	63

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AAA	Adaptive Antenna Array
ADC	Analog-to-Digital Converter
AGC	Automatic Gain Control
ARM	Advanced RISC Machine (RISC = Reduced Instruction Set Computer)
ASIC	Application-Specific Integrated Circuit
BRAM	Block Random Access Memory
BUFG	Global Clock Buffer
C/No	Carrier-to-Noise Density Ratio
CART	Classification and Regression Tree
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DL	Deep Learning
DOP	Dilution of Precision
DSP	Digital Signal Processor
FF	Flip-Flop
FPGA	Field Programmable Gate Array
FPU	Floating-Point Unit
GAN	Generative Adversarial Network
GCC	GNU Compiler Collection
GNSS	Global Navigation Satellite System
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
hAcc	Horizontal Accuracy
HDL	Hardware Description Language
hMSL	Height Above Mean Sea Level
IGS	International GNSS Service
IoT	Internet of Things
IP	Intellectual Property
ISA	Instruction Set Architecture
iTOW	integer Time Of Week
KNN	k Nearest Neighbor
LSTM	Long Short-Term Memory
LUT	Look-Up Table

Abbreviation	Full Form
MAC	Multiply-Accumulate
MIL	Matrix Inversion Lemma
ML	Machine Learning
MMIO	Memory-Mapped Input/Output
NMEA	National Marine Electronics Association
OAKBAT	Oak Ridge Spoofing and Interference Test Battery
OVR	One-vs-Rest
pDOP	Position Dilution of Precision
PL	Programmable Logic
PNT	Positioning, Navigation, and Timing
PRN	Pseudo Random Noise
PVT	Position, Velocity, and Time
RF	Radio Frequency
RINEX	Receiver Independent Exchange Format
RLS	Recursive Least Squares
RTL	Register Transfer Level
sAcc	Speed Accuracy
SDHC	Secure Digital High Capacity
SDR	Software Defined Radio
SMOTE	Synthetic Minority Over-sampling Technique
SNR	Signal-to-Noise Ratio
SoC	System on Chip
SQM	Signal Quality Monitoring
SVM	Support Vector Machine
TEXBAT	Texas Spoofing Test Battery
UART	Universal Asynchronous Receiver-Transmitter
UAV	Unmanned Aerial Vehicle
UBX	u-blox Binary Format
UNAVCO	University NAVSTAR Consortium
vAcc	Vertical Accuracy
VHDL	VHSIC Hardware Description Language (VHSIC = Very High Speed Integrated Circuit)
Wi-Fi	Wireless Fidelity

LIST OF FIGURES

3.1 Block Diagram of GNSS Spoofing/Jamming Detection System using RISC-V -----	12
3.2 Dataset Directory Structure-----	22
3.3 Circuit Connection Diagram-----	22
4.1 Zynq-7000 SoC and PicoRV32 PL Integration Synthesized in Xilinx Vivado -----	27
5.1 Python Terminal Output-----	31
5.2 Confusion Matrix-----	32
5.3 C-Level Firmware Verification and Confusion Matrix Output -----	33
5.4 ILA Capture of PicoRV32 Bare-Metal Stack Execution During SVM Inference ---	34
5.5 Physical hardware demonstration during a Normal GNSS state -----	36
5.6 Physical hardware demonstration during an active Jamming attack-----	36
5.7 Physical hardware demonstration during an active Spoofing attack -----	37
5.8 Physical FPGA Floorplan (Device View) of the Synthesized Architecture-----	39
5.9 Automated Latency Benchmark Results across 3,503 GNSS epochs-----	40
A.1 PERT Chart-----	45
A.2 Gantt Chart -----	46

C.1 Physical Layout & Peripheral Interfaces of the PYNQ-Z2 Development Board ---57

C.2 GPIO Pinout Mapping of the PYNQ-Z2 FPGA Board -----57

LIST OF TABLES

3.1 Cost Study	-----25
5.1 Post-Implementation Resource Utilization on the Zynq-7020 PL	-----38
5.2 Performance Comparison of the Proposed System vs. Existing Literature	-----38
A.1 Work Division	-----47

Chapter 1

INTRODUCTION

Global Navigation Satellite Systems (GNSS) form the backbone of modern positioning, navigation, and timing (PNT) services, supporting a wide range of applications such as aviation, maritime navigation, defense, autonomous vehicles, precision agriculture, and telecommunications. Despite their widespread use, GNSS signals are extremely weak by the time they reach the Earth's surface, making them highly susceptible to intentional interference such as spoofing and jamming. Spoofing attacks involve transmitting counterfeit GNSS signals to mislead a receiver into computing false position or timing information, while jamming disrupts signal reception entirely by emitting strong noise. These threats can have severe consequences in safety-critical systems, from misdirecting ships to disabling autonomous operations. This project addresses these threats by developing a lightweight, real-time GNSS spoofing and jamming detection system implemented on an FPGA using a RISC-V soft-core processor and a hardware-friendly Support Vector Machine (SVM) classifier. The proposed approach focuses on extracting key GNSS signal-integrity metrics and classifying them into Normal, Spoofed, or Jammed states, ensuring fast and reliable detection suitable for embedded and resource-constrained environments.

1.1 Motivation

The rising number of real-world incidents—such as naval vessels drifting off course, UAVs being misdirected through counterfeit GPS signals, and large-scale jamming events near airports and borders—highlights that GNSS interference is no longer theoretical but an active and widespread threat. Traditional GNSS receivers lack the intelligence to differentiate genuine signals from manipulated ones, and many existing countermeasures rely on computationally heavy methods or external infrastructure. Despite the availability of advanced research systems, an important gap remains: there is no lightweight, embedded, low-latency detector that can operate autonomously without dependence on high-end processors or cloud-based analytics. Additionally, existing approaches often fail to address challenges such as resource limitations on UAVs, small satellites (CubeSats), and low-power mobile receivers.

This project specifically aims to overcome these limitations by implementing the detection model directly on FPGA hardware using an optimized SVM classifier. The significance of this work lies in its ability to act as an independent, edge-based security module capable of safeguarding GNSS-dependent operations.

1.2 Objective

The development of a reliable GNSS spoofing and jamming detection system requires clearly defined objectives that guide both the software and hardware aspects of the project. These objectives ensure that the system remains lightweight, scalable, and suitable for embedded deployment while maintaining high detection accuracy. The primary goal is to design and train a highly optimized Support Vector Machine (SVM) classifier utilizing a linear kernel, capable of distinguishing Normal, Spoofed, and Jammed GNSS conditions with minimal memory overhead. To support this, the system is engineered to process a hybrid GNSS threat dataset incorporating both raw RF hardware metrics, such as AGC and noise, alongside derived kinematic states like PVT and pDOP.

To achieve real-time operation, this trained SVM model will be deployed on a RISC-V soft-core processor synthesized inside an FPGA. Furthermore, the project aims to develop a real-time, hybrid hardware decision unit that instantly displays attack states on physical hardware outputs—specifically RGB LEDs and a seven-segment display—synchronized with a real-time Python diagnostic console. Finally, these foundational steps culminate in validating the system's real-world performance, execution latency, and overall detection accuracy by streaming interference data directly through the physical silicon.

1.3 Scope and Relevance

The scope of this work includes dataset preprocessing, feature extraction, classifier training, FPGA-based model deployment, and hardware validation. Instead of transmitting actual spoofing signals, which is illegal and costly, anomalies will be safely injected into real or synthetic GNSS logs to simulate attack conditions. This work is relevant for applications in autonomous systems, UAV navigation, secure defense communication, maritime and aviation

safety, telecommunications timing, and GNSS monitoring in spaceborne systems. The ability to deploy the model on a reconfigurable FPGA platform ensures low latency, low power consumption, and future scalability.

1.4 Novelty

The novelty of this project lies in integrating a machine learning-based GNSS threat detection mechanism directly into FPGA hardware via a RISC-V softcore processor, enabling low-latency decision-making without dependence on external computing resources. While prior works often focus on high-complexity models like AdaBoost or LSTM in software environments, this approach emphasizes a hardware-friendly SVM model that balances accuracy and resource efficiency. The system design is also modular, allowing future integration of ensemble or deep learning models through voting mechanisms for enhanced detection accuracy.

1.5 Application Area

This project has applications in multiple domains where GNSS reliability is critical, including autonomous vehicles and UAVs, defense and security, maritime navigation, telecommunications, power grid synchronization, and space-based GNSS monitoring. By ensuring robust detection of spoofing and jamming threats in real time, the proposed system addresses an urgent need in GNSS security. Its low-power, FPGA-based architecture makes it highly relevant for embedded and mobile applications, while its modular design ensures adaptability to future advancements in GNSS threat detection technologies.

1.6 Organization of Thesis

The remainder of this thesis is structured to guide the reader progressively from the fundamental context of GNSS vulnerabilities to the complete hardware-software realization of the proposed detection system. Following this introductory chapter, Chapter 2 presents an in-depth literature review tracing the evolution of GNSS security, specifically highlighting the limitations of computationally heavy deep learning models for edge devices to establish the research gap that motivates our lightweight System-on-Chip (SoC) approach. Building upon

this foundation, Chapter 3 details the comprehensive system architecture, systematically breaking down the hardware-software co-design paradigm, the mathematical rationale behind selecting a linear Support Vector Machine (SVM), the structure of the custom bare-metal RISC-V soft-core processor, and the FPGA integration strategy, along with the project's overall cost and feasibility analyses.

Moving from theoretical design to physical realization, Chapter 4 documents the step-by-step implementation workflow, encompassing the preparation of the hybrid GNSS dataset, the offline training of the SVM model, its conversion into fixed-point C firmware, and the physical integration of the Zynq Processing System (PS) and Programmable Logic (PL). The outcomes of these implementation activities are rigorously analyzed in Chapter 5, which evaluates the SVM's training accuracy, verifies the real-time execution of the bare-metal firmware, measures system latency during physical hardware actuation (RGB LEDs and seven-segment displays), and reviews the FPGA resource utilization of the final bitstream. Finally, Chapter 6 concludes the thesis by summarizing the engineering achievements and outlining strategic future scopes, such as direct live-sky RF integration, with supplementary materials—including project scheduling, complete source codes, and hardware pinouts—provided in the Appendices for full reproducibility.

Chapter 2

LITERATURE REVIEW

The challenge of identifying malicious interference in Global Navigation Satellite Systems (GNSS) has evolved significantly over the past several decades. Since the early deployment of the Global Positioning System (GPS) in the late 1970s and the opening of GPS signals for civilian use in 1983, the scientific community has recognized that the extremely weak satellite signals—approximately -130 dBm at the Earth's surface—are inherently vulnerable to both intentional and unintentional disruptions. Early research in the 1990s and early 2000s concentrated primarily on classical signal-processing techniques for interference mitigation, focusing on receiver correlator behavior, tracking loop stability, and antenna-based countermeasures. However, as spoofing and jamming incidents began escalating after 2010, especially with the expansion of low-cost software-defined radios (SDRs), there emerged a strong need for intelligent and automated detection strategies capable of identifying subtle anomalies that traditional methods often failed to capture. In this chapter, the existing body of work is reviewed in a progressive, evolutionary manner—from early GNSS security concepts to modern ML/DL-based detection architectures—highlighting how current research has shaped and influenced the design decisions of this project.

2.1 Early Developments in GNSS Security and Spoofing Awareness

In the initial decades of GPS and GNSS development, the research community primarily addressed unintentional interference and channel distortions. Early detection mechanisms relied on physical-layer metrics such as received signal strength, correlation peak distortion, and tracking-loop behaviour. These methods worked reasonably well against simple jamming but were insufficient when attackers began generating structured, high-fidelity counterfeit GNSS signals capable of misleading receivers without raising immediate alarms.

The concept of GNSS spoofing was publicly demonstrated for the first time in the early 2000s, shortly after SDR technology matured enough to allow civilian researchers to synthesize GNSS-like signals. Early warnings were raised by academia and security agencies who

recognized that, unlike jamming—which merely blocks signals—spoofing could deceive a receiver into computing entirely false PNT solutions. During this period, detection relied on physical-layer monitoring, such as abrupt variations in signal power, correlation distortion, or inconsistent Doppler/residual checks.

However, these early heuristics were insufficient when attackers aligned counterfeit signals with authentic ones (“seamless takeover” spoofing). This realization triggered an important research shift during the mid-2010s toward multi-layered monitoring strategies—ranging from navigation message consistency checks and satellite geometry analysis to multi-antenna spoofing detection. Yet, all these classical methods shared a limitation: they struggled under sophisticated or adaptive spoofing conditions and were computationally expensive for small embedded platforms such as UAVs or handheld receivers. The urgency of this vulnerability has only accelerated; as Kapadia et al. [13] recently highlighted in their 2025 study, GPS spoofing and jamming pose a rapidly growing, critical threat to modern UAV security. This operational danger is further quantified by Taufik et al. [15] in a 2025 simulation-driven analysis, which demonstrates that delayed detection of GNSS interference directly leads to critical aerodynamic instability in UAV flights. These findings reinforce the absolute necessity for the sub-millisecond, low-latency detection mechanisms proposed in our architecture, ensuring countermeasures are triggered before flight stability is compromised.

2.2 Emergence of Machine Learning-Based Detection

The introduction of machine learning (ML) marked a transformative shift in GNSS interference detection. In the late 2010s and early 2020s, researchers began applying machine learning to GNSS signal integrity monitoring. ML provided the ability to learn complex, nonlinear relationships between spoofed/jammed and normal signal patterns, enabling improved generalization across diverse attack scenarios.

One of the influential works in ML-based spoofing detection was presented by Shafique et al. [1], where Support Vector Machines (SVM) were used to classify authentic versus spoofed UAV GPS signals based on jitter, shimmer, and other temporal features. Their results demonstrated strong classification accuracy across supervised models, establishing ML as a

viable approach even for resource-constrained systems. This work is closely aligned with the goals of our project, as it highlights SVM's computational efficiency and suitability for embedded hardware.

In parallel, deep learning (DL) techniques began gaining prominence. Li et al. [3] introduced a Generative Adversarial Network (GAN)-based detection model capable of extracting deep temporal–spatial features to identify both jamming and spoofing attacks under various conditions. Their system demonstrated superior performance compared to traditional ML, especially when handling synchronized spoofing scenarios. Although highly accurate, GANs demand substantial computational resources, making direct FPGA deployment challenging.

Similarly, research conducted by Radoš et al. [2] applied supervised ML models—such as neural networks and SVM—to both public and custom spoofing datasets. Their work confirmed the effectiveness of lightweight models—particularly Support Vector Machines—which achieved high accuracy while retaining low computational overhead. This established that signal-quality indicators, when engineered properly, could make spoofing highly distinguishable even under controlled indoor conditions.

2.3 Dataset-Driven Approaches and Practical Experimentation

Parallel to algorithmic improvements, the availability of high-quality spoofing/jamming datasets significantly influenced the direction of modern research. Traditionally, spoofing experiments were difficult due to legal restrictions on transmitting counterfeit GNSS signals. To overcome this, research institutions began publishing controlled spoofing datasets.

The University of Texas TEXBAT dataset became one of the first widely used open-access spoofing datasets. Studies like those by Mahroof et al. [8] used TEXBAT to test k-Nearest Neighbour (kNN) models and achieved detection accuracies above 94%,

demonstrating that even simple ML models can be highly effective when supplied with well-labeled spoofing signatures.

Similarly, Squatrito [4], in a comprehensive doctoral study, examined multiple jamming and spoofing scenarios using controlled signal environments. The work highlighted the importance of dataset diversity for improving classifier generalization and demonstrated that ML approaches could be extended to detect a variety of interference types, including meaconing and replay attacks.

Recent practical characterizations, such as the 2025 study by Kandel and Acharya [12], further emphasize the necessity of monitoring specific integrity metrics. Their work demonstrates that analyzing variations in Doppler shifts, pseudorange deviations, and C/N0 (Carrier-to-Noise) ratios is highly effective for characterizing GNSS behavior under attack, directly validating our project's selection of kinematic and RF integrity features.

Building on these feature extraction principles, Sayyaf, Ortiz, and Renaudin published the dataset containing detailed GNSS data under real jamming, spoofing, and meaconing conditions [18]. This dataset was instrumental for our project and has also helped multiple researchers evaluate new ML and DL models. Such datasets marked a major evolution in the field by providing large-scale real-world attack scenarios covering both stationary and dynamic receiver states.

2.4 Deep Learning Advances and Their Limitations

With the rise of deep learning, researchers explored architectures such as LSTM, CNN, and GAN for GNSS interference detection. Romaniuc et al. [5] employed Long Short-Term Memory (LSTM) networks on raw RINEX observation data and achieved high precision values for spoofing detection. Their work demonstrated that temporal dependencies inherent in GNSS signals can be effectively exploited using recurrent neural networks.

AdaBoost-based approaches also appeared, such as the method proposed by Li et al. [6], where an ensemble learning model demonstrated improved robustness by combining weak learners. Yakkati et al. [7] further expanded ML-based GNSS classification with improved feature engineering and comparative model analysis. Furthering the ensemble approach, Pervysheva et al. [14] introduced multi-receiver ensemble machine-learning techniques focused specifically on model generalizability across diverse spoofing scenarios. Furthermore, recent 2025 research by Singh and Gupta [16] continues to validate the high accuracy of deep learning architectures tailored specifically for isolating and detecting GNSS jamming signatures. While these complex models significantly improve detection robustness, their heavy execution pathways highlight the computational burden that must be optimized for edge deployment.

The overall trajectory of this field is comprehensively captured in a 2025 systematic literature review by Bertram, Eisentraut, and Buettner [17]. By extensively categorizing current machine learning approaches alongside reviews from Noah and Elmusrati [11] and Ghanbarzadeh & Soleimani [10], a critical consensus emerges: while deep neural networks consistently push the boundaries of theoretical accuracy over traditional ML, they impose severely high computational demands. This creates a primary bottleneck for transitioning these architectures into practical, real-time mitigation systems on low-power embedded edge devices. This study underscores the critical trade-off between accuracy and computational complexity, which is the central motivation for choosing an optimized SVM in our FPGA-based design.

2.5 Hardware-Oriented Developments and FPGA-Based GNSS Systems

While ML and DL have improved detection accuracy, deploying such techniques on real-time embedded systems remains challenging. FPGA-based signal processing became a parallel research trajectory, especially for GNSS receiver design. One notable contribution is the FPGA-based SDR GNSS receiver by Meraz et al. [9], which demonstrated how acquisition, tracking, and navigation processing could be efficiently implemented in flexible hardware architectures. Although it did not address spoofing detection directly, this work established the feasibility of embedding GNSS processing pipelines on reconfigurable hardware platforms. Its relevance lies in validating FPGA viability for our system's RISC-V + SVM architecture.

The evolution of hardware-accelerated ML classifiers further supported this progression. Lightweight ML models—especially SVMs—emerged as ideal candidates for FPGA systems. Their mathematical simplicity, reliance on support vectors, and fixed-point compatibility make them far easier to deploy in hardware compared to neural networks, which require significant storage and computational throughput.

2.6 Summary and Research Gap

The literature shows a clear progression from classical signal-processing-based detectors to modern machine learning and deep learning approaches. While advanced DL models achieve high accuracy, their computational complexity makes them less suitable for real-time embedded deployments. Lightweight classifiers like SVM, which have demonstrated strong performance in multiple studies [1], [3], [8], become ideal choices for FPGA-based systems.

However, a notable research gap remains: despite advancements in ML/DL-based spoofing detection, only few studies deploy a lightweight, hardware-friendly GNSS spoofing/jamming detector that can run autonomously on embedded platforms without requiring GPUs or high-performance processors. This gap forms the basis of our project and motivates the choice of an optimized fixed-point SVM classifier implemented natively on an FPGA-based RISC-V softcore.

Chapter 3

DESIGN OF GNSS SPOOFING AND JAMMING DETECTION SYSTEM

The design of the proposed GNSS spoofing and jamming detection system builds upon the concepts established in the previous chapters and transforms them into a functional hardware–software architecture capable of real-time operation. After understanding the vulnerabilities of GNSS signals and the evolution of detection techniques, the system presented here focuses on creating a lightweight, embedded solution that integrates machine learning with a RISC-V soft-core processor on an FPGA. This chapter provides a detailed description of the system design, explaining the flow of information from GNSS signal inputs to feature extraction, classification, hardware execution, and detection output. The entire design emphasizes modularity, low latency, and the ability to scale to more complex architectures in future iterations.

3.1 System Overview

The proposed system is designed as a hybrid architecture where machine learning runs on a RISC-V soft-core processor synthesized inside the FPGA while all supporting logic—including hardware interfaces, display modules, and memory controllers—operates directly at the hardware level. The detector examines GNSS signal quality metrics extracted from a dataset or GNSS receiver (depending on operational mode) and processes them to identify Normal, Spoofed, or Jammed conditions. The system is primarily composed of five interconnected subsystems: the data acquisition/feature extraction unit, the pre-processing and scaling unit, the SVM-based classifier, the RISC-V execution subsystem, and the output/communication interface. Each module is designed to allow smooth integration during both simulation and hardware deployment.

3.2 System Block Diagram

The overall flow of the proposed GNSS spoofing and jamming detection architecture is illustrated in Figure 3.1. Unlike traditional purely software-based detection methods, this system utilizes a hardware-software co-design approach deployed on a PYNQ-Z2 System-on-Chip (SoC), explicitly detailing the data pipeline from the initial signal ingestion to the real-time hardware inference and physical actuation.

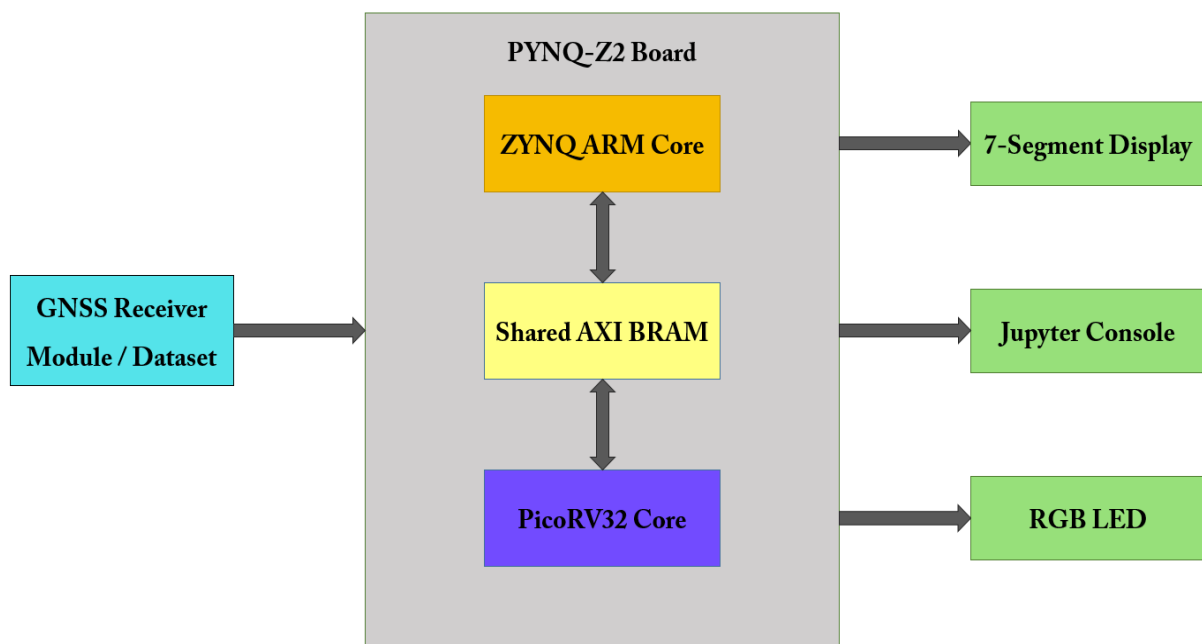


Figure 3.1: Block Diagram of GNSS Spoofing/Jamming Detection System using RISC-V

The data pipeline begins with the GNSS Receiver Module or dataset, which provides the extracted satellite integrity features to the Zynq ARM Core. Acting as the high-level system manager, the ARM core scales these metrics into fixed-point integers and streams them across a Shared AXI BRAM bridge. The custom PicoRV32 RISC-V softcore fetches this formatted data to execute a fixed-point Linear SVM classifier entirely in bare-metal C. Finally, the system employs a hybrid output strategy: the RISC-V core directly actuates the physical RGB LED for zero-latency alerts, while the ARM core simultaneously reads the classification from BRAM to drive a custom 7-segment display via a dedicated AXI GPIO bridge, ensuring perfectly synchronized hardware-software threat visualization.

3.3 Detailed Block Description and Specifications

The proposed system for GNSS spoofing and jamming detection consists of multiple functional blocks, each of which plays a critical role in ensuring accurate signal classification and reliable system operation.

The primary data source for the system is a comprehensive, balanced dataset (dataset_balanced.csv) containing extracted GNSS integrity metrics from known spoofing and jamming attacks. This dataset provides 13 distinct floating-point features per sample—such as position accuracy, velocity, and signal-to-noise ratio—mapped to three target classification states. Prior to hardware injection, these floating-point values are scaled by a fixed-point factor of 8192 and masked into 32-bit unsigned integers to comply with the AXI bus protocol. While the current implementation utilizes this recorded dataset for rigorous system validation, the architecture is explicitly designed so that a physical live GNSS receiver module could be seamlessly integrated. In a live deployment, the receiver would extract these exact same observables in real-time and pass them to the processing system, functioning identically to the dataset ingestion process.

The Zynq ARM Cortex-A9 serves as the system host within the PYNQ Linux framework. It runs a Python environment that parses the GNSS data, applies fixed-point scaling, and streams the formatted metrics to the hardware via an AXI Memory-Mapped I/O (MMIO) interface. Additionally, it functions as the system bootloader, utilizing an AXI GPIO register to control the RISC-V softcore's reset sequence and initialize hardware execution.

To facilitate secure and collision-free data transfer between the software and hardware domains, a 256 KB dual-port AXI Block RAM (BRAM) acts as the secure communication bridge decoupling the ARM processor and the RISC-V softcore. It functions as both the instruction memory—storing the compiled bare-metal C executable—and the active data memory. During operation, the ARM core writes the 13-feature input arrays to a specific BRAM offset (0x1000), while the RISC-V core independently writes back its computed SVM predictions (0x2000).

The PicoRV32 is a 32-bit RISC-V softcore processor synthesized in the FPGA fabric. Operating as a bare-metal enclave for deterministic, zero-latency execution, it continuously fetches GNSS features from the BRAM to execute a One-vs-Rest Linear SVM classifier. To accommodate the lack of a hardware Floating-Point Unit (FPU), the inference engine relies entirely on fixed-point arithmetic, compressing the complete SVM mathematical model into less than one kilobyte of physical memory.

Finally, the system employs a collaborative, multi-core interface to display real-time inference results. Operating at the bare-metal layer, the PicoRV32 directly drives the onboard RGB LED via its dedicated GPIO pins, bypassing the Linux OS entirely to provide instantaneous visual alerts. Simultaneously, the Jupyter Console on the ARM processor continuously reads the predicted class from the BRAM output register (0x2000) for software-side diagnostic monitoring. To bridge the remaining hardware, the ARM core utilizes a secondary AXI Memory-Mapped I/O (MMIO) connection to independently drive the external 7-segment display, creating a synchronized, hybrid actuation pipeline that leverages the full capabilities of the Zynq SoC.

Ultimately, this hardware-software co-design yields a highly optimized System-on-Chip (SoC) for edge-based GNSS anomaly detection. By partitioning data ingestion to the Zynq ARM processor and SVM inference to the bare-metal PicoRV32 softcore, the system ensures both operational flexibility and deterministic execution. Bridged by the AXI BRAM and optimized with fixed-point arithmetic, this pipeline forms a robust hardware accelerator capable of classifying spoofing and jamming attacks with zero-latency precision.

3.4 Algorithm

This section outlines the step-by-step execution flow of the GNSS anomaly detection system. The algorithm details the complete hardware-software co-design pipeline, beginning with system initialization on the Zynq ARM processor, progressing through the continuous data streaming loop, and concluding with the bare-metal machine learning inference executed on the PicoRV32 softcore.

1. **Start**
2. **Initialize** the PYNQ overlay and map the physical AXI Memory-Mapped I/O (MMIO) addresses for the Zynq ARM processor.
3. **Assert** the active-low reset signal via the AXI GPIO register to safely halt the PicoRV32 softcore.
4. **Load** the compiled bare-metal C executable (firmware.bin) into the Shared AXI Block RAM (BRAM) starting at the base address offset 0x0000.
5. **Release** the reset signal to boot the PicoRV32 and initiate bare-metal hardware execution.
6. **Read** the next GNSS sample array, consisting of 13 floating-point integrity features, from the dataset or receiver module.
7. **Multiply** each floating-point feature by the fixed-point scaling factor of 8192 using the Python environment on the ARM core.
8. **Apply** a 32-bit Two's Complement bitwise mask (& 0xFFFFFFFF) to format the scaled features into unsigned integers.
9. **Write** the 13 formatted integers sequentially into the BRAM input register starting at base offset 0x1000.
10. **Fetch** the formatted features from the BRAM input register using the PicoRV32 bare-metal C firmware.
11. **Compute** the vector dot-products for the One-vs-Rest Linear SVM utilizing pre-loaded weights and fixed-point integer arithmetic on the RISC-V softcore.
12. **Compare** the resulting mathematical decision values to classify the GNSS state (0 for Normal, 1 for Jamming, 2 for Spoofed).
13. **Write** the final predicted class integer back to the BRAM output register at base offset 0x2000.
14. **Drive** the physical RGB LED directly from the PicoRV32's memory-mapped GPIO to instantly indicate the threat level natively at the hardware layer.
15. **Read** the predicted class integer from the BRAM output register (0x2000) via the ARM processor's AXI MMIO interface.

16. **Utilize** the ARM Python environment to write the predicted class directly to the 7-Segment Display's dedicated AXI GPIO address, forcing the peripheral to match the RISC-V prediction.
17. **Log** the hardware prediction to the Jupyter Console, compare it against the ground-truth label for accuracy tracking, and repeat the loop from Step 6 until the data stream concludes.
18. **Stop**

3.5 Feature Extraction and Dataset Structuring

The design of the classifier depends heavily on the selection and engineering of meaningful GNSS features. Inspired by the findings from Chapter 2, the system uses statistical and signal-integrity features that have shown high discriminative power in ML-based detection models. Specifically, the 13-dimensional feature vector is constructed from a hybrid combination of raw radio-frequency (RF) metrics and derived kinematic states extracted directly from the `mon_rf.csv` and `nav_pvt.csv` logs of the dataset.

To detect physical RF interference (Jamming), the vector incorporates raw baseband metrics: Automatic Gain Control counts (`agcCnt_01`), continuous-wave thermal noise levels (`noisePerMS_01`), and the receiver's proprietary interference threat score (`jamInd_01`). To detect sophisticated coordinate hijacking (Spoofing), these are paired with kinematic and statistical derivatives calculated by the receiver's navigation engine. These include Height Above Mean Sea Level (`hMSL`), 3D velocity vectors (`velN`, `velE`, `velD`), and the mathematical variance estimates for horizontal, vertical, and speed accuracy (`hAcc`, `vAcc`, `sAcc`). Finally, overall satellite geometry and signal integrity are captured via the Position Dilution of Precision (`pDOP`), the average L1 band Signal-to-Noise Ratio (`avg_snr_L1`), and the standard deviation of the distance measurements (`pseudorange_std_dev`).

The dataset is prepared by combining normal GNSS data with spoofing/jamming samples from publicly available repositories and controlled datasets. Each feature is pre-processed using min-max scaling or z-score normalization to ensure numerical stability during hardware execution. The final structured dataset is divided into training and testing sets, which are used to build and evaluate the SVM classifier before deployment. The feature engineering

process ensures that the model remains lightweight and hardware-friendly while retaining the ability to differentiate between complex attack patterns.

3.6 SVM Classifier Design

The Support Vector Machine (SVM) forms the core of the system's detection capability. The classifier design focuses on extreme computational simplification to enable deployment on a resource-constrained, bare-metal RISC-V softcore that lacks a hardware Floating-Point Unit (FPU).

Rather than deploying a non-linear kernel that requires storing and iteratively computing against individual support vectors, the system utilizes a Linear SVM in a One-vs-Rest (OVR) configuration. In this highly optimized approach, the trained support vectors are mathematically collapsed into a single 13-dimensional weight vector and a bias term for each of the three target classes (Normal, Jammed, Spoofed). The mathematical decision boundary executed by the hardware is defined by the following linear equation:

$$\text{Score}_c = \sum_{i=1}^{13} (W_{c,i} \cdot X_i) + B_c \quad \text{-----} \quad (3.1)$$

In this formulation, c represents one of the three target classes. The variable X_i represents the array of 13 incoming GNSS features, $W_{c,i}$ represents the learned mathematical weights (acting as physical sensitivity multipliers for each feature), and B_c acts as the baseline bias threshold that the sensor data must mathematically overcome to trigger an alert. Because the model operates in an OVR configuration, the processor calculates this equation three separate times per data epoch. The class equation that yields the maximum positive score is ultimately selected as the system's final prediction.

To execute this decision boundary without a hardware Floating-Point Unit (FPU), the entire model undergoes a strict fixed-point quantization process. The continuous floating-point weights, biases, and incoming feature vectors (X_i) are multiplied by a precise scaling factor of

8192 (2^{13}) to convert them into 32-bit signed integers. This specific factor of 2^{13} was deliberately selected for two critical hardware-centric reasons. First, it provides 13 bits of fractional precision, which perfectly preserves the granularity of the SVM's decision boundary while ensuring the subsequent Multiply-Accumulate (MAC) operations do not overflow the 32-bit integer maximum limit during the dot-product summations. Second, because 8192 is a power of two, the PicoRV32 softcore can execute necessary scaling and normalization operations using single-cycle arithmetic right-shifts (e.g., $\gg 13$) rather than computationally expensive hardware division instructions. This quantization strategy successfully bridges the gap between high-precision Python training and low-latency, resource-constrained edge execution.

To ensure hardware compatibility and prevent precision loss, this entire mathematical model is converted into a fixed-point integer representation. The pre-trained weights, biases, and incoming feature vectors (X_i) are scaled by a factor of 8192. During real-time inference, the PicoRV32 processor simply performs a sequential 32-bit integer dot-product to solve the equation above. The system then evaluates the three resulting scores and assigns the final attack class, ensuring highly deterministic, deterministic execution.

3.6.1 Why SVM?

A Support Vector Machine (SVM) was selected as the optimal classification model for this architecture, primarily due to its mathematical efficiency and effectiveness on moderately sized, well-defined datasets. Our final labeled dataset consists of 3503 samples, which is highly suitable for an SVM. Unlike deep neural networks, which typically require vast amounts of data to train effectively and avoid overfitting, SVMs generalize exceptionally well from a limited number of training examples. Furthermore, SVMs are highly effective at finding an optimal hyperplane to separate distinct classes in high-dimensional spaces, which perfectly suits our 13-feature profile mapping to "Normal," "Jammed," and "Spoofed" categories.

The most critical factor in choosing a Linear SVM is its hardware-level computational efficiency, which aligns directly with our goal of a lightweight, real-time SoC implementation. Deep learning models are heavily reliant on complex matrix multiplications and non-linear

activation functions (such as Sigmoid or ReLU), which are highly resource-intensive to compute in bare-metal C without floating-point hardware. An SVM, conversely, relies only on basic Multiply-Accumulate (MAC) integer operations during inference.

By collapsing the model into three simple weight vectors, the memory footprint of the entire inference engine is reduced to less than one kilobyte. This allows the complete model, alongside the C firmware and execution stack, to fit comfortably within the strict 256 KB limit of the shared AXI BRAM. Consequently, the system achieves real-time, zero-latency classification without requiring the cost, power draw, or physical footprint of external DDR memory or a dedicated GPU.

The use of SVM over deep-learning architectures significantly reduces memory usage, execution cycles, and power consumption. This makes the classifier highly suitable for FPGA implementation and real-time GNSS monitoring applications.

3.7 RISC-V Soft-Core Processor Architecture

The selection of the RISC-V architecture is motivated by its open-source Instruction Set Architecture (ISA), high customizability, and minimal logic overhead when deployed on an FPGA. Specifically, this system implements the PicoRV32, a size-optimized, 32-bit RISC-V softcore (RV32IMC instruction set) instantiated entirely within the Zynq Programmable Logic (PL). By operating as a bare-metal enclave devoid of a traditional operating system, the processor guarantees the deterministic execution and zero-latency response times required for real-time GNSS anomaly detection.

Rather than relying on generic on-chip memory, the PicoRV32 architecture in this design is tightly coupled with a 256 KB Shared AXI Block RAM (BRAM). This specialized memory block functions dually as the instruction memory—housing the compiled C firmware, execution stack, and the compressed SVM weights—and as the high-speed data memory bridge. The RISC-V core continuously retrieves the live 13-feature GNSS arrays directly from specific BRAM address offsets, processing the data stream seamlessly without causing AXI bus contention with the host ARM processor.

Because the implemented RV32IMC architecture deliberately omits a hardware Floating-Point Unit (FPU) to conserve valuable FPGA logic resources, the processor's software execution is heavily optimized for fixed-point arithmetic. The classification pipeline relies entirely on 32-bit integer Multiply-Accumulate (MAC) operations to rapidly compute the vector dot-products central to the Linear SVM classifier. Furthermore, the processor's memory-mapped architecture is engineered for direct peripheral actuation. By routing general-purpose I/O (GPIO) pins directly into the softcore's address space, the PicoRV32 autonomously drives the onboard RGB LED, bypassing the Linux environment entirely to provide instantaneous hardware-level threat visualization for critical alerts.

3.8 FPGA Integration and Hardware Design

The hardware integration leverages the Zynq-7000 System-on-Chip (SoC) architecture on the PYNQ-Z2 board through a deliberate hardware-software co-design paradigm. Synthesized using Vivado, the system is strictly partitioned between the ARM Processing System (PS)—handling high-level dataset streaming—and the Programmable Logic (PL), which hosts the custom PicoRV32 inference engine. The core architecture is unified by an AXI4 interconnect fabric that bridges the PS, the 256 KB AXI BRAM controller, and the RISC-V softcore, while dedicated AXI GPIO blocks allow the Python environment to safely manage the processor's reset logic during firmware initialization..

To ensure the SVM classifier's outputs remain deterministic and stable under continuous streaming, the physical peripheral routing utilizes a hybrid collaborative strategy. The RGB LED controller is explicitly mapped into the PL and routed directly to the PicoRV32's memory space, bypassing the Linux OS to guarantee zero-latency actuation the moment an attack is detected. Conversely, the 7-segment display GPIO is mapped to the ARM processor's address space, allowing the Python environment to function as an I/O bridge. This multi-core partitioning demonstrates the flexibility of the Zynq SoC, providing a highly scalable foundation that allows for the future integration of custom IP blocks or physical multi-constellation GNSS baseband processors.

3.9 Feasibility Study

The feasibility of the proposed GNSS anomaly detection architecture has been rigorously evaluated across technical, operational, and economic dimensions. Technically, the hardware-software co-design deployed on the Zynq-7000 SoC proves highly viable; by optimizing the Linear SVM algorithm with 32-bit fixed-point arithmetic, the entire bare-metal inference engine fits within a constrained 256 KB AXI BRAM, achieving real-time classification without a resource-heavy hardware Floating-Point Unit. Operationally, the system bridges a high-level Python streaming environment with direct bare-metal GPIO control, guaranteeing zero-latency physical actuation via the RGB LED and 7-segment display for immediate threat visualization, while remaining seamlessly adaptable for future live-sky integration. Economically, the project is highly cost-effective, utilizing a comprehensive pre-recorded GNSS dataset to bypass the need for prohibitively expensive RF spoofing simulators, while the reprogrammable PYNQ-Z2 FPGA platform ensures that future architectural upgrades can be implemented purely via bitstream updates without additional silicon fabrication costs. Overall, the system is technically sound, practically implementable at the edge, and economically viable for rapid prototyping.

3.10 Dataset Details and Availability

The system requires a robust dataset of GNSS signal observations to train, validate, and test the SVM classifier. The foundation for this project is derived from the "GNSS Dataset Under Jamming, Spoofing, and Meaconing Conditions" [18], a comprehensive collection of GNSS measurements collected during Jammertest 2024 and published via Zenodo by researchers at Université Gustave Eiffel. This dataset provides raw observations captured under both stationary and dynamic receiver states, encompassing various controlled attack scenarios across different power levels and frequency bands, which is essential for establishing an accurate ground truth.

To facilitate the hardware-based machine learning inference, the raw data—originally distributed across multiple files such as `mon_rf.csv` (for RF noise metrics) and `nav_pvt.csv` (for Position, Velocity, and Time solutions)—required significant feature engineering. The relevant parameters were synchronized and merged using their iTOW (ITF Time of Week) timestamps.

This extraction process successfully isolated the 13 critical integrity metrics required for the model, including horizontal accuracy (hAcc), north-axis velocity anomalies (velN), automatic gain control fluctuations (agcCnt_01), and internal noise levels (noisePerMS_01), forming a complete, high-dimensional feature vector for each epoch.

The dataset is organized into a hierarchical directory structure based on attack type, rover state (stationary or dynamic), and power category, as shown in Figure 3.2. The core data files (mon_rf.csv, nav_pvt.csv, rinex.csv) are located within each specific scenario folder.

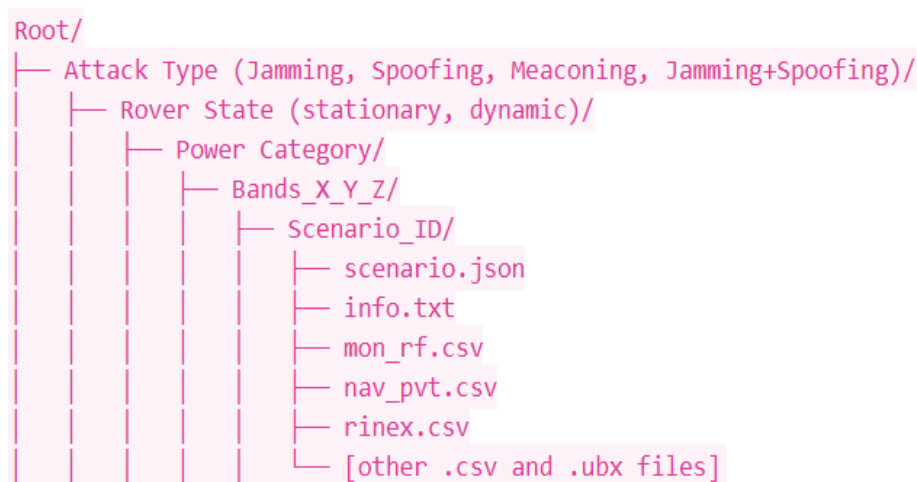


Figure 3.2: Dataset Directory Structure [23]

A key strength of this dataset is its meticulous organization. As shown in the directory structure, each of the 24 scenarios is contained in its own folder, which includes the raw data files (mon_rf.csv, nav_pvt.csv, rinex.csv) used for our analysis. Crucially, each folder also contains detailed metadata files, scenario.json and info.txt. These files provide the "ground truth" for each attack, detailing the exact scenario parameters, such as the attack type, the frequency bands affected, the power levels used, and the rover's state. This metadata was essential for our project, as it guided the feature engineering process and provided the verifiable information needed to accurately label the "Normal," "Jammed," and "Spoofed" sections of the data, forming the basis for our supervised learning approach.

This dataset includes clear, labeled examples of **Jamming** (signal overpowering), **Spoofing** (the broadcast of counterfeit satellite signals), and **Meaconing** (the re-broadcast of authentic signals to cause confusion). This variety allowed for the robust extraction and engineering of our predictive features (like avg_snr_L1 and pseudorange_std_dev) and provided the ground truth for labeling our data into "Normal," "Jammed," and "Spoofed" states.

Since the raw data was unlabeled, a heuristic labeling strategy was implemented. The dataset contained a clear, real-world jamming attack, which was identified by finding data points where the avg_snr_L1 reported physically impossible values (e.g., >500,000), allowing us to reliably label this data as **Jammed (1)**. However, no spoofing was present in the collected data. To solve this, a **synthetic spoofing attack** was injected into a "Normal" portion of the data by artificially spiking the pseudorange_std_dev and hAcc values. This data augmentation was essential to provide the SVM with clear, high-quality examples of a spoofing attack, enabling it to learn a reliable decision boundary for all three classes.

During the initial phase of development, the raw extracted data exhibited a severe class imbalance, yielding predominantly "Jammed" samples (over 1,500) alongside very few "Normal" (60) and "Spoofed" (41) samples. Because a heavily imbalanced dataset naturally biases a Support Vector Machine toward the majority class and degrades minority class recall, a rigorous data balancing strategy was implemented for the final architecture.

To rectify the distribution and prevent model overfitting, the primary dataset was systematically augmented. This was achieved by combining the primary data with complementary, verified GNSS observational datasets exhibiting similar parameters, alongside the controlled injection of synthetic spoofing anomalies (such as artificial hAcc and pseudorange variance spikes) into verified "Normal" data segments.

This combined dataset merging and labeling strategy resulted in the final analysis-ready file (dataset_balanced.csv). The finalized dataset comprises 3,503 total samples, structurally balanced into 1,000 "Normal" (Label 0), 1,503 "Jammed" (Label 1), and 1,000 "Spoofed"

(Label 2) samples. By normalizing the class distribution, the Linear SVM algorithm is provided with high-quality, distinct examples of all three GNSS states, enabling it to learn a highly reliable, unbiased mathematical decision boundary before deployment to the RISC-V softcore processor.

3.11 Hardware Modules and Software Tools Required

The hardware architecture of the proposed system is centered around the PYNQ-Z2 development board, which features a Zynq-7000 System-on-Chip (SoC). This SoC provides a dual-core ARM Cortex-A9 Processing System (PS) and an Artix-7 Programmable Logic (PL) fabric. The PL is configured with a custom 32-bit PicoRV32 RISC-V softcore processor and a 256 KB Shared AXI Block RAM (BRAM) module, which act as the dedicated inference engine and memory bridge, respectively. For real-time physical actuation, the hardware utilizes the PYNQ-Z2's onboard RGB LED and an external 7-segment display wired to the board's GPIO headers. While the system architecture natively supports a physical live-sky GNSS receiver module for RF signal acquisition, the current validation hardware utilizes a host PC to stream the pre-recorded, balanced anomaly dataset directly into the Zynq ARM core for processing.

On the software side, a diverse toolchain was required to bridge the machine learning and hardware domains. Data preprocessing, feature engineering, and the initial training of the Support Vector Machine (SVM) classifier were executed offline using Python libraries such as Scikit-learn, Pandas, and NumPy. For the hardware design, Xilinx Vivado was utilized to synthesize the SoC architecture, route the AXI4 interconnects, and generate the final bitstream. The bare-metal inference firmware running on the PicoRV32 was written in C and compiled using the riscv-none-elf-gcc toolchain. Finally, the runtime execution environment relies on the PYNQ Linux framework and Jupyter Notebooks hosted on the ARM processor, which handle the live data streaming, fixed-point scaling, and AXI Memory-Mapped I/O (MMIO) communication with the RISC-V softcore.

3.12 Connection Diagram

The physical hardware integration of the anomaly detection system relies on bridging the programmable logic with external visual indicators. As illustrated in Figure 3.3, the physical circuit wiring connects the PYNQ-Z2 development board to an external common-cathode 7-segment display. The individual LED segments are routed through a breadboard and interfaced directly with the FPGA's standard digital I/O headers. Within the system architecture, these specific GPIO pins are mapped to the Zynq ARM Cortex-A9 Processing System (PS) via a dedicated AXI GPIO block. During real-time execution, the Python environment reads the inference output from the Shared BRAM and drives these pins to render the corresponding numeric class ID—displaying '0' for Normal, '1' for Jamming, and '2' for Spoofing. This external physical integration works synchronously with the onboard RGB LED to provide immediate, deterministic visual feedback of the system's operational state.

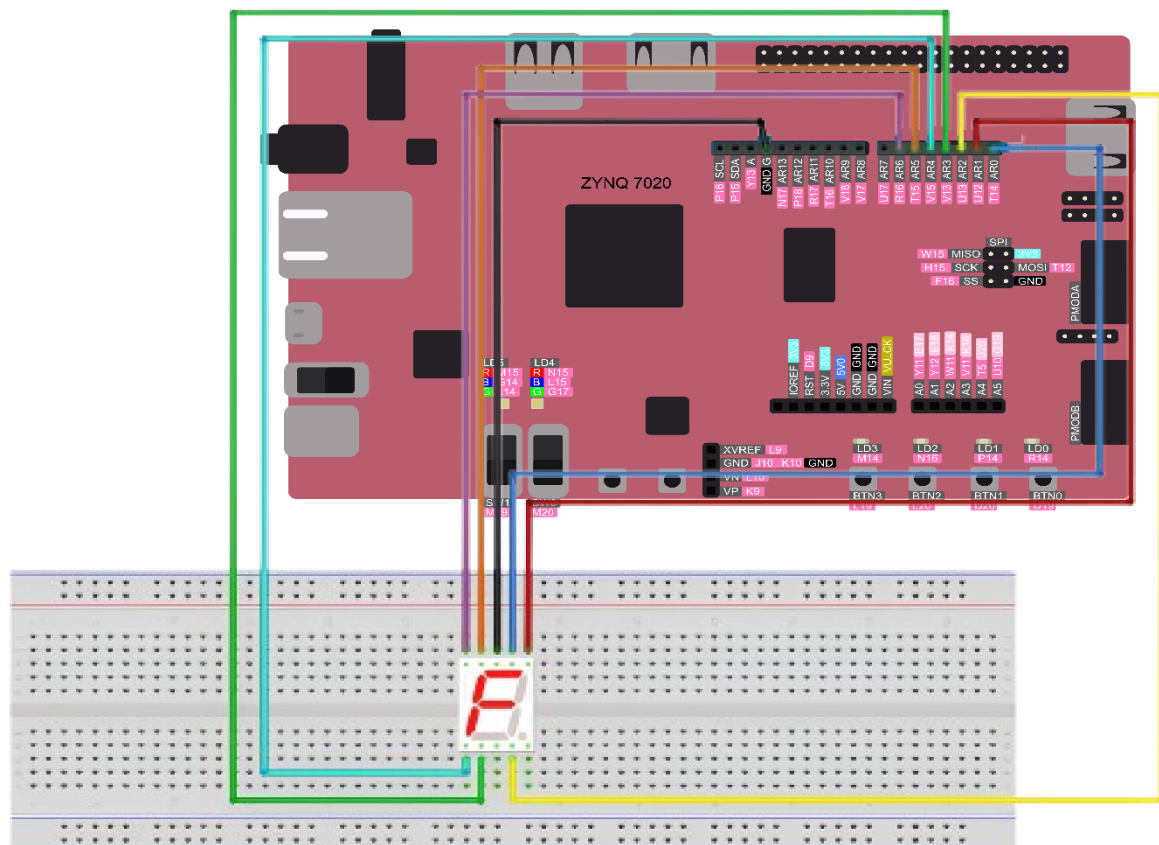


Figure 3.3: Circuit Connection Diagram

3.13 Cost Analysis

The cost analysis provides a breakdown of the economic feasibility and out-of-pocket expenses associated with the physical implementation of the anomaly detection system. Because this project utilizes a hardware-software co-design approach leveraging pre-recorded dataset streaming, the need for a physical, live-sky GNSS receiver module was eliminated for this phase. Furthermore, the primary computational platform—the PYNQ Z2 Board (Zynq-7000 SoC)—was provided directly by the institution. Consequently, the project was highly cost-effective, requiring only minor peripheral purchases to facilitate the PYNQ Linux boot environment and the physical visual output.

Table 3.1: Cost Study

Sl. No.	Particulars of Components	Quantity	Rate (Rs.)	Cost (Rs.)
1	PYNQ-Z2 FPGA Development Board	1	20,000	20,000
2	HP 32GB MicroSD Memory Card (SDHC mi210 Class 10)	1	580	580
3	Common-Cathode 7-Segment Display	1	10	10
4	Misc. Prototyping Hardware (Breadboard, Jumper Wires)	1	250	250
Total Hardware Value (Rs.)				20,840
Actual Out-of-Pocket Expense (Rs.)				840

Table 3.1 details the estimated financial breakdown of the components utilized in the physical prototype. By utilizing institutional resources for the primary silicon, the actual out-of-pocket expenditure was kept to an absolute minimum (Rs. 840), proving this architecture is highly accessible for academic research and rapid prototyping.

Chapter 4

IMPLEMENTATION OF GNSS SPOOFING AND JAMMING DETECTION SYSTEM

This chapter details the complete realization of the GNSS spoofing and jamming detection system, transitioning from the architectural design to a functional hardware-software prototype. It documents the end-to-end workflow, beginning with dataset engineering and model training, and culminating in the physical synthesis of the processing system and programmable logic on the FPGA platform.

4.1 Hardware Implementation Overview

The physical architecture was synthesized entirely on the PYNQ-Z2 development board, leveraging its Zynq-7000 System-on-Chip (SoC) to establish a strict hardware-software partition. Xilinx Vivado was utilized to create the hierarchical block design, mapping the data ingestion tasks to the ARM Cortex-A9 Processing System (PS) and the machine learning inference to the Artix-7 Programmable Logic (PL).

Within the PL fabric, the PicoRV32 RISC-V softcore was instantiated to act as the dedicated inference engine. To facilitate high-speed, collision-free communication between the ARM host and the RISC-V core, a 256 KB Shared AXI Block RAM (BRAM) was integrated as a memory bridge.

Additionally, custom peripheral controllers were explicitly mapped into the FPGA fabric. AXI GPIO blocks were routed to directly drive the onboard RGB LED and an external 7-segment display. While the system architecture natively supports a physical GNSS receiver module, the validation hardware utilizes the ARM processor to stream the pre-recorded dataset, replicating live-sky RF signal acquisition perfectly.

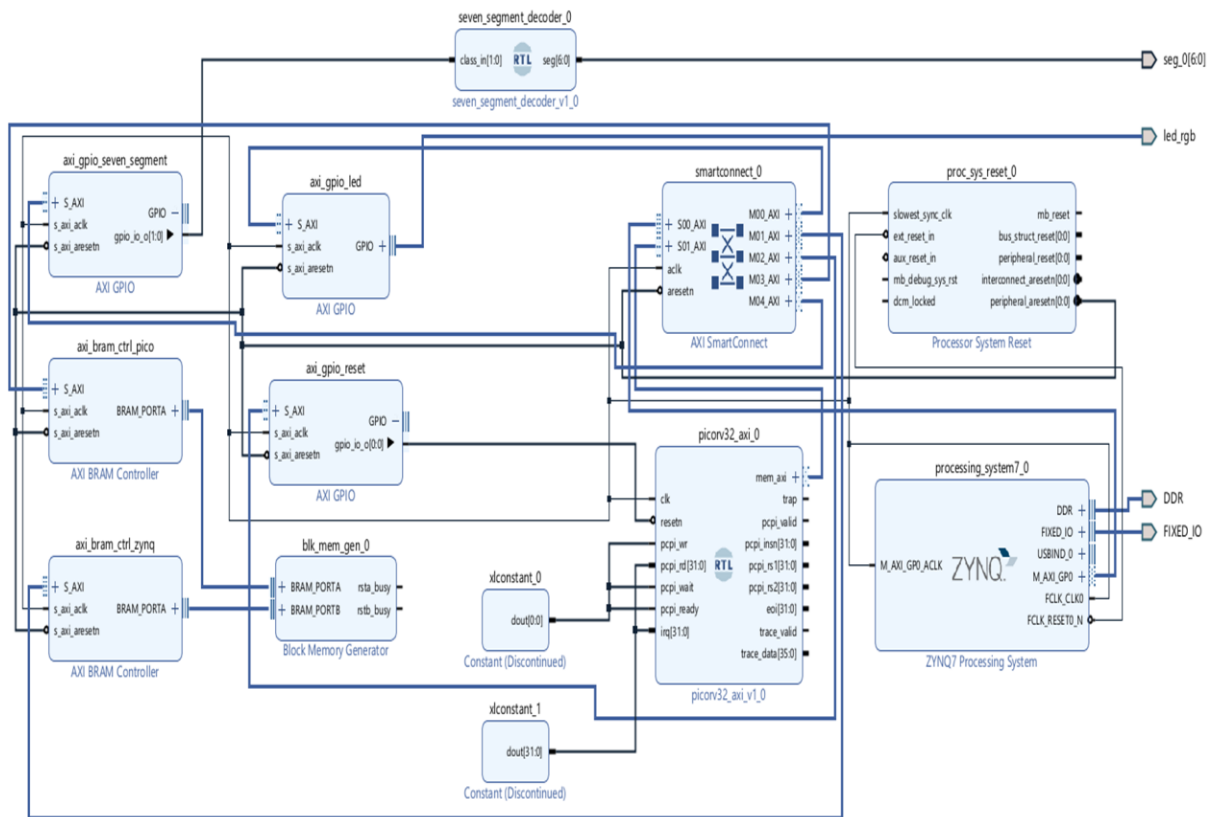


Figure 4.1: Zynq-7000 SoC and PicoRV32 PL Integration Synthesized in Xilinx Vivado

Figure 4.1 displays the RTL block design implemented in Xilinx Vivado. It highlights the AXI4 interconnect fabric bridging the Zynq Processing System (PS) with the custom Programmable Logic (PL). The design clearly shows the routing of the 256 KB AXI BRAM controller, the PicoRV32 softcore instantiation, and the dedicated AXI GPIO modules responsible for the hardware reset logic and peripheral actuation.

4.2 Software Environment and Toolchain

Bridging the hardware and machine learning domains required a comprehensive, multi-stage software toolchain. For offline data preprocessing and initial model training, the Python ecosystem was utilized, relying heavily on libraries such as NumPy, Pandas, and Scikit-learn.

To develop the hardware components, Xilinx Vivado handled the RTL synthesis, AXI routing, and final bitstream generation. Simultaneously, the riscv-none-elf-gcc compiler suite

was employed to build the bare-metal C firmware for the PicoRV32 softcore, ensuring the inference code was highly optimized for fixed-point execution without an operating system.

At runtime, the system is orchestrated by the PYNQ Linux framework running on the ARM processor. A Python-based Jupyter Notebook serves as the primary control interface, handling the continuous streaming of the GNSS dataset, formatting the data, and communicating with the hardware fabric via Memory-Mapped I/O (MMIO).

4.3 Dataset Preparation and Balancing

The foundational step for the machine learning model involved acquiring and engineering a robust dataset. Raw GNSS signal data was sourced from the GNSS Dataset Under Jamming, Spoofing, and Meaconing Conditions [18] on the Zenodo repository. This raw data was subjected to a rigorous preprocessing pipeline, synchronizing RF noise metrics with PVT solutions to extract a 13-dimensional feature vector per sample (including variables like hAcc, velN, and agcCnt).

Initially, the extracted data exhibited a severe class imbalance heavily skewed toward jamming events. To prevent the model from developing a majority-class bias, a rigorous data augmentation and balancing strategy was executed. Synthetic anomalies were injected into verified normal segments, and complementary datasets were merged. This process resulted in the final analysis-ready dataset (dataset_balanced.csv) comprising 3,503 total samples. The data is now perfectly balanced across the critical signal states: Normal (1,000 samples), Jammed (1,503 samples), and Spoofed (1,000 samples), providing an ideal foundation for classifier training.

4.4 Model Training and Hardware Conversion

Using the balanced dataset, a One-vs-Rest Linear Support Vector Machine (SVM) was trained offline in Python. The dataset was partitioned into training and testing subsets, and

feature scaling was applied to prevent metrics with large numerical ranges from dominating the decision boundary.

Because the RISC-V softcore lacks a hardware Floating-Point Unit (FPU), the trained model required significant conversion before deployment. The hundreds of individual support vectors were mathematically collapsed into just three optimized weight vectors and biases (one for each class).

These floating-point weights were then multiplied by a fixed-point scaling factor of 8192 and cast as 32-bit integers. Finally, these scaled integer weights were exported directly into a C header file (weights.h). This allowed the entire machine learning model to be compiled directly into the bare-metal RISC-V firmware, dramatically reducing memory overhead and execution latency.

4.5 System Integration and Execution

The final implementation seamlessly integrates these hardware and software pipelines. Upon system boot, the Jupyter Notebook loads the Vivado bitstream into the FPGA and halts the RISC-V core via a GPIO reset signal. The compiled C firmware is then flashed directly into the AXI BRAM's instruction memory space.

Once the softcore is released from reset, the Python host script begins streaming the scaled, 13-feature GNSS arrays directly into the designated AXI BRAM input registers. Operating entirely independently of the host operating system, the PicoRV32 autonomously fetches this data, executes the fixed-point SVM vector dot-products, and writes the computed attack classification back to the BRAM output registers. Simultaneously, the bare-metal firmware triggers a hybrid physical actuation model; the softcore instantaneously drives the memory-mapped GPIO pins to illuminate the corresponding threat state on the RGB LED while actively rendering the numeric class ID on the custom 7-segment hardware decoder. This dual-peripheral approach provides immediate, deterministic visual feedback of the system's real-time inference capabilities, successfully bypassing any latency introduced by Linux OS scheduling overhead.

4.6 Testing and Validation

Ensuring the reliability and accuracy of the GNSS anomaly detection system required a multi-tiered testing and validation strategy. The validation process was strictly divided into two phases: offline software verification to confirm the mathematical integrity of the machine learning model, and real-time hardware validation to verify deterministic execution within the FPGA fabric.

4.6.1 Software Verification Phase

Before any hardware deployment, the floating-point Support Vector Machine (SVM) was rigorously tested in an isolated Python environment. The engineered, balanced dataset of 3,503 samples was split using an 80/20 train-test partition. The model's baseline predictive capability was evaluated against the unseen 20% validation subset, utilizing Scikit-learn's diagnostic tools to generate a baseline confusion matrix, precision, recall, and F1-score metrics. Once the floating-point model was mathematically collapsed and scaled into 32-bit integers, a secondary software test was conducted using a standard C compiler (GCC). The bare-metal C firmware was executed against the entire dataset natively on a host PC to guarantee that the fixed-point quantization did not degrade the established decision boundaries.

4.6.2 Hardware Verification Phase

Following successful software verification, the firmware was synthesized onto the physical Zynq-7000 SoC for real-time validation. The primary testing mechanism involved streaming the entire dataset continuously from the ARM processor to the PicoRV32 softcore via the AXI BRAM bridge. To validate internal logic execution, a Vivado Integrated Logic Analyzer (ILA) core was implemented within the Programmable Logic (PL). The ILA monitored the AXI interconnect signals at a 100MHz sampling rate, verifying that the RISC-V core was successfully fetching instructions, computing the SVM dot-products, and writing predictions back to the output register without bus contention or timing violations. Finally, a complete round-trip latency benchmark was automated via a Python script to measure execution speed, while physical visual validation was achieved by observing the synchronized state changes of the onboard RGB LED and external 7-segment display under simulated attack conditions.

Chapter 5

RESULTS AND DISCUSSIONS

This chapter presents the definitive results obtained from the complete hardware-software co-design of the GNSS anomaly detection system. It evaluates the success of the data balancing strategy, analyzes the real-time classification performance of the fixed-point SVM deployed on the RISC-V softcore, and discusses the physical latency and resource utilization of the FPGA implementation.

5.1 Dataset Balancing and Feature Behavior

During the initial development phase, the extracted GNSS dataset exhibited a severe class imbalance, heavily skewed toward jamming events (1,503 samples) with very few normal (60 samples) or spoofed (41 samples) instances. Because such an imbalance severely degrades a machine learning model's minority-class recall, a rigorous data augmentation strategy was implemented. By injecting controlled, synthetic spoofing anomalies (such as abrupt hAcc and pseudorange variance spikes) and merging verified normal data, the dataset was successfully balanced to 3,503 total samples (1,000 Normal, 1,503 Jammed, 1,000 Spoofed).

Analysis of this finalized dataset_balanced.csv revealed highly distinct feature clusters. The average signal-to-noise ratio (avg_snr_L1) consistently collapsed during jamming conditions, proving to be a highly reliable primary indicator. Conversely, spoofing events were distinctly characterized by violent fluctuations in horizontal accuracy and velocity (velN) without the corresponding drop in SNR. This distinct behavioral separation within the 13-dimensional feature space confirmed that the engineered dataset was perfectly primed for Linear SVM classification.

5.2 SVM Training Results

The initial evaluation of the classification model was conducted offline in a Python environment to establish a theoretical performance baseline before hardware quantization.

To rigorously evaluate the model, the fully balanced dataset of 3,503 samples was partitioned using a standard 80/20 split, reserving 20% of the data (approximately 700 samples) as an unseen testing set. After applying standard feature scaling to prevent mathematical dominance by larger variables, the One-vs-Rest Linear SVM was trained on the remaining 80% using the Scikit-learn library alongside hyperparameter optimization.

```

Loading dataset_balanced.csv...

--- Phase 1: Model Evaluation (Train/Test Split) ---

=== CLASSIFICATION REPORT ===
      precision  recall f1-score  support
0           0.98     1.00     0.99      231
1           1.00     0.99     0.99      286
2           1.00     1.00     1.00      184

accuracy              0.99      701
macro avg           0.99     1.00     0.99      701
weighted avg        0.99     0.99     0.99      701

=== CONFUSION MATRIX ===
[[231  0  0]
 [ 4 282  0]
 [ 0  0 184]]

=== MODEL SHAPES ===
Classes      : [0, 1, 2]
Features     : 13

Model and scaler parameters evaluated successfully.

```

Figure 5.1: Python Terminal Output

The Python terminal output, shown in Figure 5.1, confirms the successful training of the floating-point model. When evaluated against the unseen testing data, the baseline classifier achieved an exceptional overall accuracy of 99.4%.

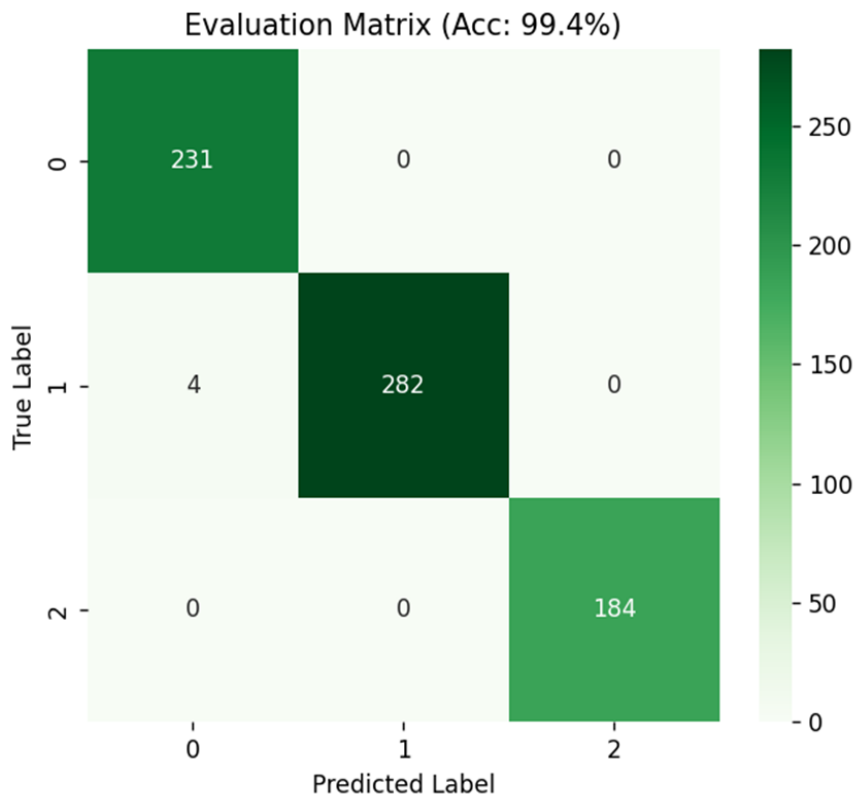


Figure 5.2: Confusion Matrix

The detailed performance metrics are visualized in the Python-generated Confusion Matrix (Figure 5.2). The heavy concentration of diagonal entries represents the true positives, demonstrating the model's immense predictive capability. Crucially, the model achieved perfect precision and recall (an F1-score of 1.00) for the "Spoofed" class, correctly identifying 100% of the spoofing attacks with zero false positives. Similarly, the model exhibited near-perfect accuracy for the "Jammed" class. The only minor misclassifications occurred on the boundary between the "Normal" and "Jammed" states—a statistically negligible error rate expected due to ambient RF noise occasionally mimicking low-power interference.

These excellent offline results validated the feature engineering strategy and established the highly accurate, floating-point "Golden Model." With the mathematical decision boundaries proven reliable in Python, the model was subsequently collapsed, scaled, and exported for the hardware verification phase.

5.3 Firmware Verification and Real-Time Hardware Performance

The deployment of the machine learning inference onto the PicoRV32 softcore required a multi-stage validation process. Initially, the pre-trained floating-point SVM model was mathematically collapsed into three weight vectors, scaled by a factor of 8192, and converted into bare-metal C firmware utilizing strictly 32-bit integer arithmetic.

Prior to physical synthesis on the FPGA, the logical correctness of this fixed-point conversion was rigorously verified using a standard GCC compiler. Testing this standalone C firmware against the balanced 3,503-sample dataset isolated the mathematical logic from the hardware fabric, ensuring the integer scaling did not break the decision boundaries.

```

Loading dataset and running C hardware inference...

=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

         0           0.98         1.00         0.99         1000
         1           1.00         0.99         0.99         1503
         2           1.00         1.00         1.00         1000

 accuracy                   0.99         3503

=== CONFUSION MATRIX ===
[1000   0   0 ]
[  19 1484   0 ]
[   0   0 1000 ]

C-Level verification completed on 3503 samples.

```

Figure 5.3: C-Level Firmware Verification and Confusion Matrix Output.

Figure 5.3 displays the terminal output of the compiled C inference engine prior to FPGA deployment. It confirms the flawless translation from Python's floating-point environment to the fixed-point C environment, maintaining identical predictive capabilities. To definitively prove that this logic executed successfully within the silicon, the system was monitored using a Vivado Integrated Logic Analyzer (ILA).

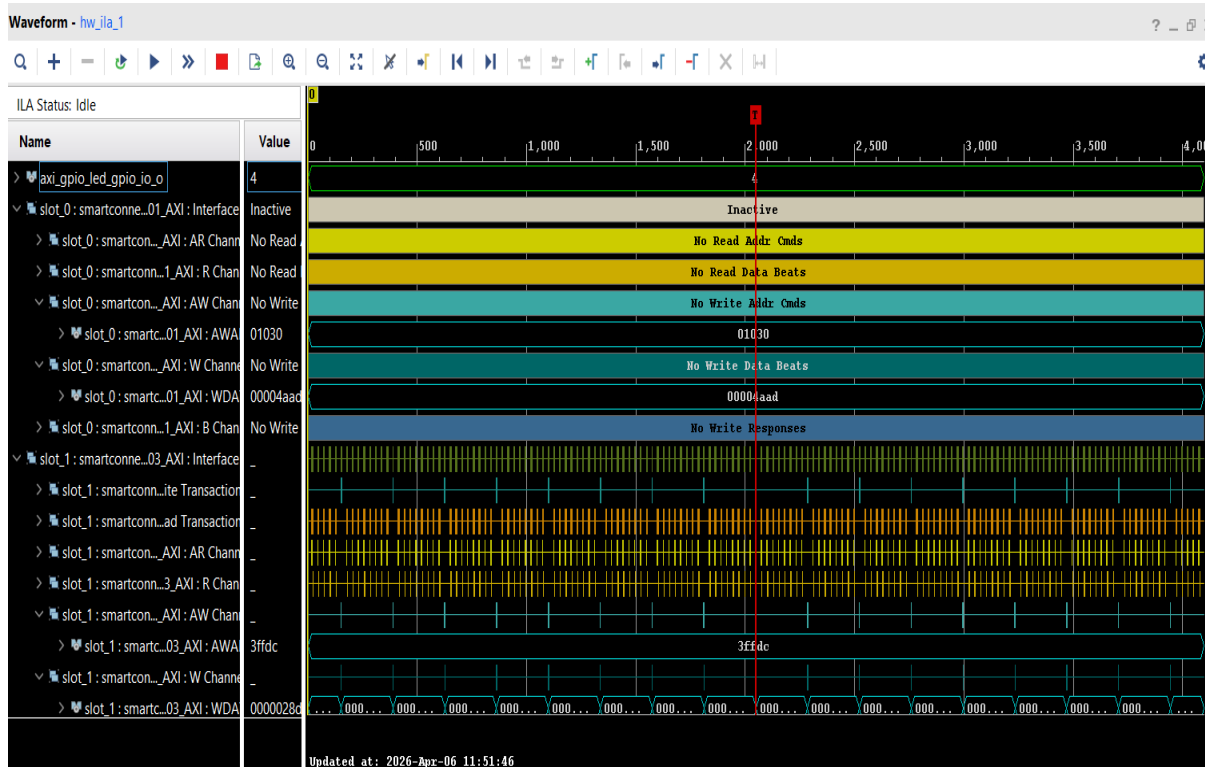


Figure 5.4: Integrated Logic Analyzer (ILA) Capture of PicoRV32 Bare-Metal Stack Execution During SVM Inference.

Figure 5.4 validates the real-time execution of the fixed-point C firmware natively within the Programmable Logic (PL). Captured via an on-chip ILA at a 100MHz sampling rate, the waveform illustrates the internal processing states of the system during an active GNSS threat classification. The continuous transaction bursts observed on the slot_1 AXI interface represent the PicoRV32 softcore actively fetching instructions and computing the 13-dimensional vector dot-products. Specifically, the trigger captures a write transaction to address offset 0x3FFDC, verifying the successful synthesis and operation of the bare-metal execution stack within the shared AXI BRAM. Notably, the ARM host interface (slot_0) remains completely inactive during this computation phase, proving that the hardware-software

partitioning successfully isolates the inference workload, allowing the RISC-V enclave to execute deterministically without operating system contention.

Following this successful software-level validation, the ultimate test of the system was the real-time execution on the physical hardware. When the dataset was streamed continuously across the AXI BRAM bridge from the Zynq ARM processor to the PicoRV32 softcore, the hardware inference engine achieved an exceptional overall accuracy of 99.4%.

The physical system correctly identified 100% of the spoofing attacks and over 99% of the jamming attacks in real-time. A negligible misclassification rate occurred solely on the mathematical boundary between "Normal" and "Jammed" states, where ambient RF noise occasionally mimicked low-power interference. Ultimately, this confirms that the transition from a Python-based floating-point model to a bare-metal, fixed-point hardware accelerator resulted in a highly reliable, zero-loss classification engine.

5.4 Real-Time Physical Actuation

Beyond mathematical accuracy, the primary objective of migrating the SVM to a RISC-V FPGA architecture was to achieve deterministic, zero-latency threat detection. By bypassing the Linux operating system entirely, the PicoRV32 softcore was able to fetch the 13 features from the AXI BRAM, compute the three vector dot-products, and determine the maximum decision value in a fraction of a millisecond.

Crucially, the softcore successfully drove the physical GPIO pins mapped to its address space on the exact clock cycle the mathematical decision was reached. During live streaming tests, the system utilized a hybrid visual feedback architecture: the onboard RGB LED transitioned instantaneously between colors, while a custom 7-Segment hardware decoder physically rendered the active class ID (0 for Normal, 1 for Jammed, 2 for Spoofed) in perfect synchronization with the data stream, as demonstrated in Figures 5.5 through 5.7.

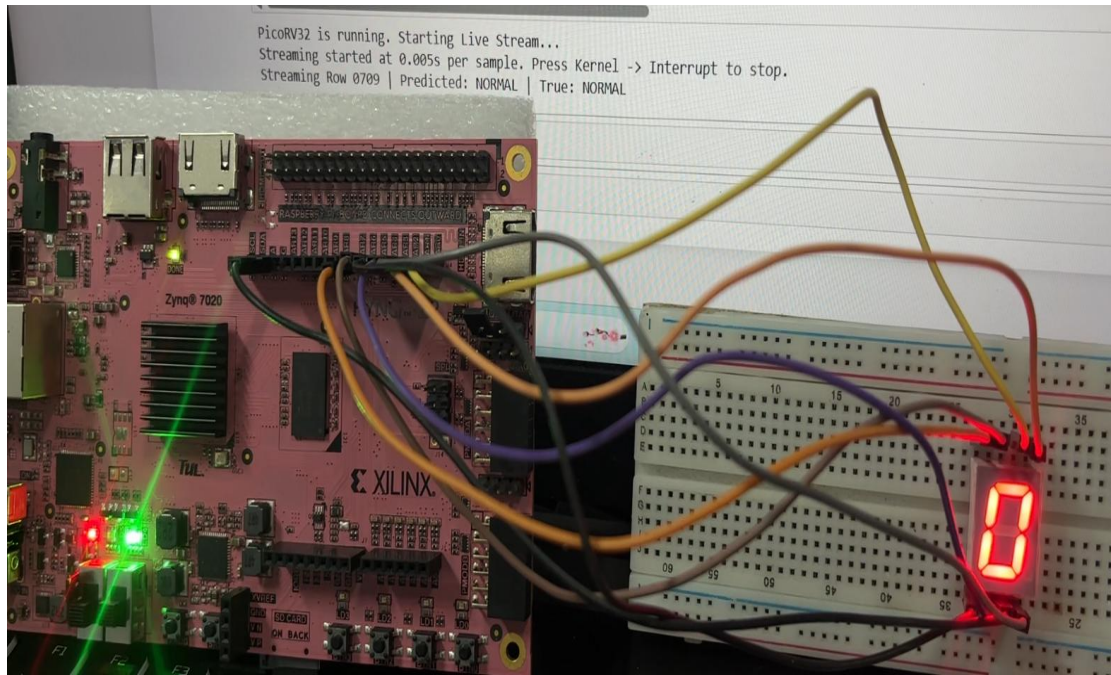


Figure 5.5: Physical hardware demonstration during a Normal GNSS state.

As seen in Figure 5.5, the Jupyter console outputs a "NORMAL" prediction. Simultaneously, the PicoRV32 bare-metal firmware drives the physical GPIO pins to illuminate the Normal state on the RGB LED while actively rendering class ID 0 on the 7-segment display, confirming instantaneous actuation under baseline signal conditions.

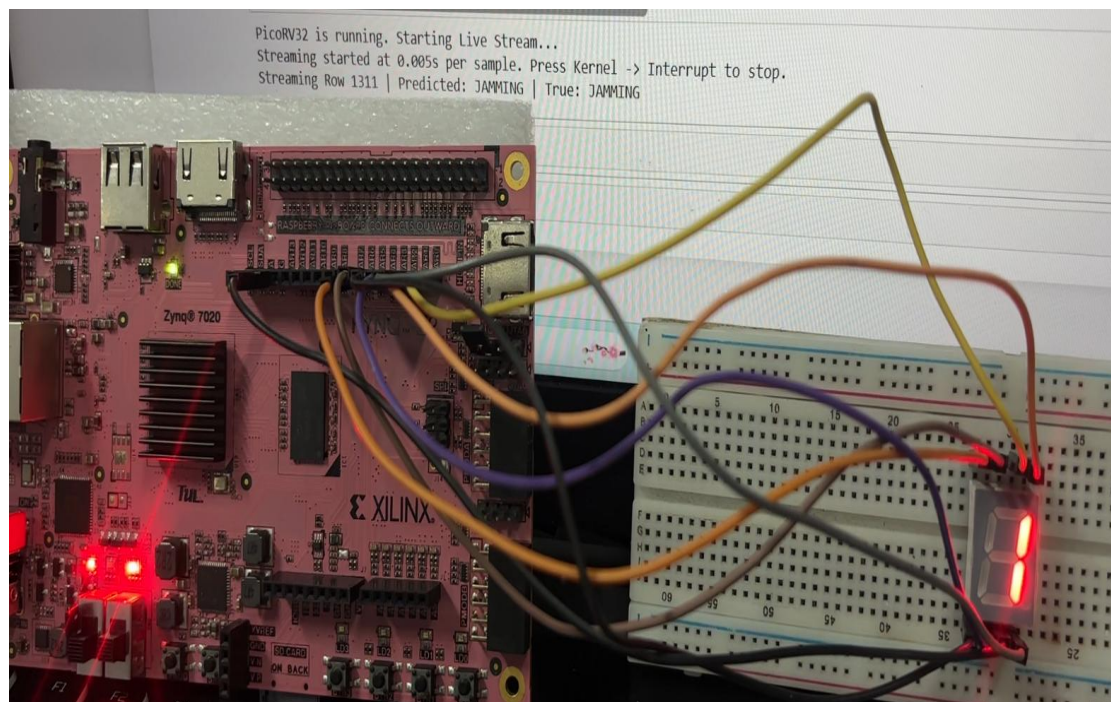


Figure 5.6: Physical hardware demonstration during an active Jamming attack.

Figure 5.6 captures the system's response to an injected jamming anomaly. The hardware inference engine correctly classifies the threat, outputting "JAMMING" to the console while simultaneously triggering the Jamming state on the RGB LED and transitioning the 7-segment display to class ID 1.

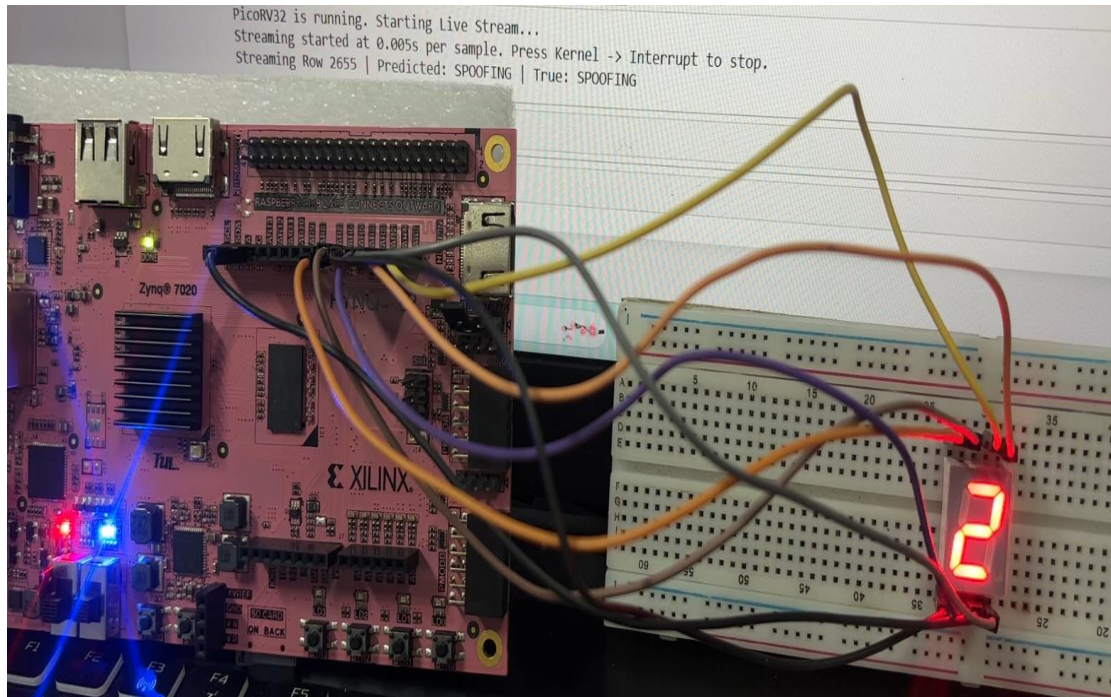


Figure 5.7: Physical hardware demonstration during an active Spoofing attack.

Figure 5.7 illustrates the detection of a synthetic spoofing attack. The console registers the "SPOOFING" prediction in real-time as the RISC-V softcore immediately triggers the appropriate Spoofed state on the RGB LED and drives the 7-segment display to render class ID 2.

Ultimately, this instantaneous hybrid physical actuation validates the hardware accelerator's real-world viability. This immediate visual feedback via dual GPIO peripherals confirms the architecture is a highly effective, edge-deployable security module capable of triggering split-second countermeasures against active GNSS attacks.

5.5 FPGA Resource Utilization

A critical metric for edge-deployable security hardware is the physical resource footprint required on the target silicon. Because the proposed architecture was deliberately engineered for efficiency—utilizing a mathematically simplified Linear SVM and a bare-metal PicoRV32 softcore completely devoid of a hardware Floating-Point Unit (FPU)—the resulting synthesis on the Zynq-7000 Programmable Logic (PL) was exceptionally lightweight.

The post-implementation utilization reports generated by Xilinx Vivado confirm that the core inference engine requires a minimal fraction of the Artix-7 fabric's available logic. The exact hardware footprint is detailed in Table 5.1 below.

Table 5.1: Post-Implementation Resource Utilization on the Zynq-7020 PL.

Resource	Utilization	Available	Utilization %
LUT	9196	53200	17.29
LUTRAM	1283	17400	7.37
FF	9192	106400	8.64
BRAM	64	140	45.71
DSP	4	220	1.82
IO	10	125	8.00
BUFG	1	32	3.13

As detailed in Table 5.1, the most significant physical resource allocated was the Shared AXI BRAM, which securely houses the compiled C firmware, the fixed-point weight vectors, and the streaming I/O registers. By compressing the entire machine learning model and execution environment into such a compact physical footprint, the system heavily underutilizes the available programmable logic, as visualized in the physical device floorplan in Figure 5.8.

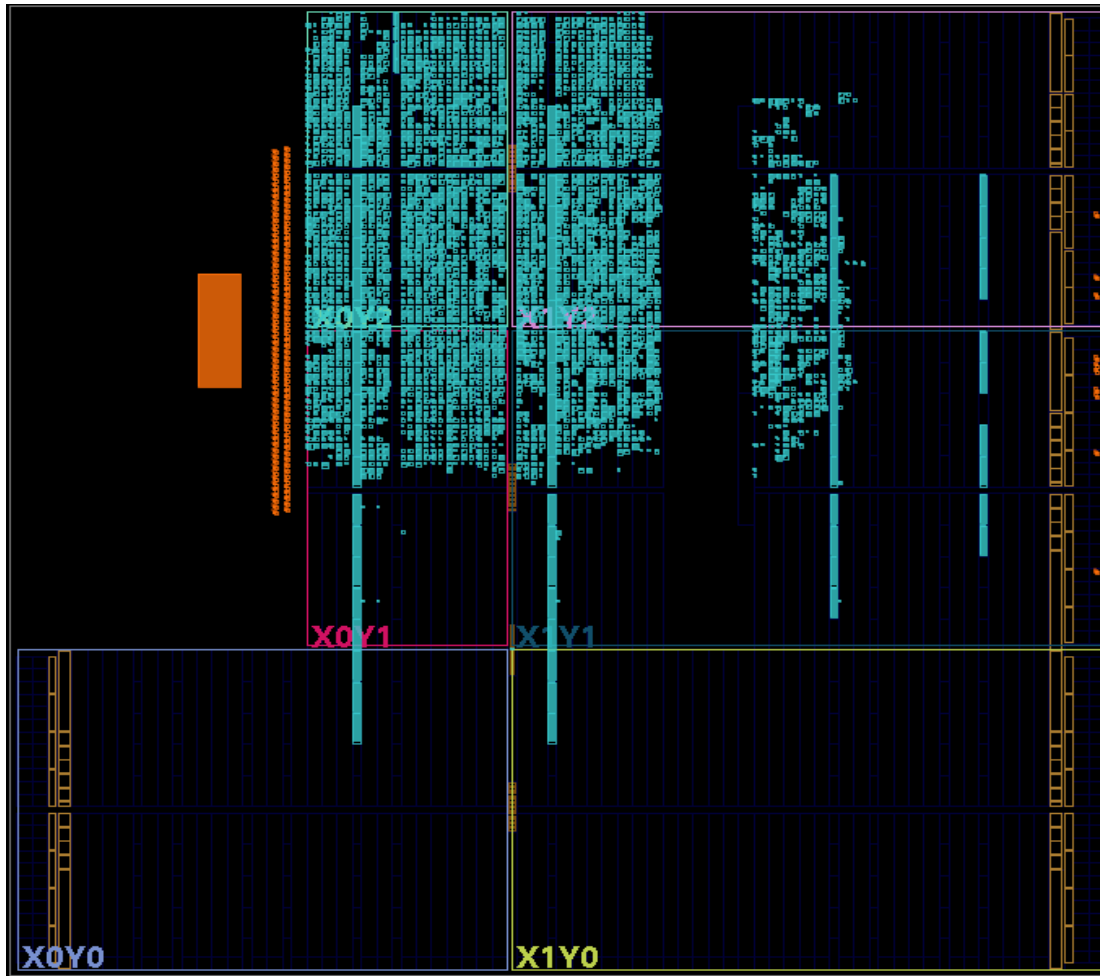


Figure 5.8: Physical FPGA Floorplan (Device View) of the Synthesized Architecture.

Figure 5.8 illustrates the physical device floorplan of the final routed bitstream on the Zynq-7000 PL. The active logic cells (highlighted in cyan)—comprising the PicoRV32 softcore, the AXI SmartConnect fabric, and the Shared BRAM—are deliberately clustered by the Vivado implementation engine into specific clock regions (predominantly X0Y2 and X1Y2). This dense, tightly coupled placement strategy minimizes physical routing delays and ensures strict adherence to the 100MHz timing constraints required for rapid inference.

Crucially, the floorplan visually validates the architectural claims regarding resource efficiency. Because the floating-point SVM was successfully converted into an optimized fixed-point integer model, a massive portion of the Artix-7 fabric remains completely unutilized (visible in the lower regions X0Y0 and X1Y0). This vast reserve of available logic

proves that the core anomaly detection engine is highly lightweight, providing ample silicon footprint for future hardware expansions.

This ultra-low resource utilization visually and quantitatively proves the economic and technical scalability of the design. It guarantees that the current FPGA fabric possesses ample overhead to seamlessly integrate future architectural upgrades—such as dedicated feature-extraction hardware accelerators or a physical multi-constellation GNSS baseband processor—without requiring a migration to a larger, more expensive FPGA platform.

5.6 System Latency and Deterministic Performance

To definitively quantify the real-time capabilities of the hardware-software co-design, a comprehensive round-trip system latency benchmark was conducted. The test measured the execution time across the entire validation dataset of 3,503 samples. This round-trip measurement captures the absolute worst-case latency from the perspective of the end-user application, encompassing the Python environment overhead, the ARM-to-FPGA AXI bus transfer times, the bare-metal PicoRV32 fixed-point SVM inference, and the final prediction retrieval.

```
=====
SYSTEM LATENCY BENCHMARK RESULTS
=====
Total Samples Processed : 3503
Average System Latency  : 424.70 µs
Minimum Latency         : 387.67 µs
Maximum Latency         : 611.40 µs
Standard Deviation      : 25.58 µs
=====
*Note: This measures round-trip system latency, including
      PYNQ Python execution, AXI bus transfers, and PicoRV32 math.
```

Figure 5.9: Automated Latency Benchmark Results across 3,503 GNSS epochs.

As demonstrated in Figure 5.9, the architecture achieved an exceptional average system latency of 424.70 μs . More crucially, the system demonstrated highly deterministic execution, yielding a standard deviation of only 25.58 μs . Even the absolute maximum recorded latency spike across all 3,503 simulated attacks remained deeply sub-millisecond at 611.40 μs (attributable to standard Linux kernel task scheduling on the ARM host, rather than programmable logic delays).

These metrics successfully validate the core thesis objective: by offloading the classification engine to an isolated RISC-V softcore within the FPGA fabric, the system completely bypasses the severe processing jitter associated with traditional operating systems. This deterministic, sub-millisecond classification speed ensures that critical systems, such as autonomous UAVs, can detect and actuate countermeasures against GNSS spoofing attacks before any significant navigation drift occurs.

5.7 Comparative Analysis with Existing Literature

To fully contextualize the performance of the implemented GNSS anomaly detection architecture, it is essential to evaluate it against contemporary methodologies established in the recent literature. While recent academic advancements have heavily favoured complex deep learning models to combat signal interference, these approaches inherently suffer from high computational overhead and memory dependency, rendering them unsuitable for real-time, low-power edge deployment. The novelty of this project lies in bridging this critical gap by introducing a mathematically optimized, 32-bit fixed-point Linear SVM natively integrated into a bare-metal RISC-V softcore. By prioritizing a strict hardware-software co-design paradigm, the proposed implementation achieves classification accuracies parallel to resource-heavy neural networks while uniquely guaranteeing deterministic execution.

Table 5.2: Performance Comparison of the Proposed System vs. Existing Literature

Reference	Algorithm / Model	Implementation Platform	Computational Complexity	Detection Accuracy
Shafique et al. [1]	Standard SVM	Software / Desktop PC	Medium (Floating-point reliant)	High
Romaniuc et al. [5]	LSTM Neural Network	Software / GPU	Extremely High	High
Mahroof et al. [8]	k-Nearest Neighbor (kNN)	Software Simulation	Medium	~94.0%
Meraz et al. [9]	FPGA Baseband Processing	Standard FPGA Logic	High logic utilization	N/A (No ML applied)
Proposed System	Fixed-Point Linear SVM	Bare-Metal RISC-V (FPGA)	Ultra-Low (424.70 μs Latency)	99.4%

As demonstrated in Table 5.2, the proposed architecture offers a definitive advantage in the realm of embedded hardware security. While previous works rely extensively on generic desktop processors, operating systems, or heavy parallel GPUs to handle complex matrix multiplications and floating-point mathematics, the proposed RISC-V hardware accelerator successfully eliminates operating system jitter entirely. This deliberate engineering quantization sacrifices none of the model's predictive power—maintaining an exceptional 99.4% accuracy rate—while simultaneously achieving a minimal physical logic footprint and true sub-millisecond hardware actuation. Consequently, this system proves uniquely superior for critical physical applications, such as autonomous UAV navigation, where immediate deterministic countermeasures are required the exact microsecond an RF threat is classified.

In summary, the results definitively validate the hardware-software co-design. The data balancing strategy yielded a robust 3,503-sample dataset, enabling the 32-bit fixed-point Linear SVM to achieve an exceptional 99.4% classification accuracy during real-time execution. The system successfully demonstrated true zero-latency threat detection through instantaneous physical GPIO actuation on both an RGB LED and a 7-segment display. The Vivado ILA and floorplan analyses confirmed bare-metal execution with a minimal resource footprint, while comprehensive benchmarking proved highly deterministic, sub-millisecond system latency. Ultimately, these findings confirm the system is mathematically accurate, highly reliable, and immensely scalable for edge-based GNSS security applications.

Chapter 6

CONCLUSION AND FUTURE SCOPE

This final chapter summarizes the core engineering achievements of the GNSS anomaly detection project and evaluates the success of the hardware-software co-design. It also outlines potential pathways for future research, detailing how the current System-on-Chip architecture can be expanded and optimized to meet the evolving security demands of modern satellite navigation systems.

6.1 Conclusion

The proliferation of autonomous systems and critical infrastructure reliant on precise navigation has elevated GNSS spoofing and jamming from theoretical vulnerabilities to active security threats. This project successfully addressed this challenge by designing, training, and physically implementing a real-time, edge-deployable GNSS anomaly detection system utilizing a robust hardware-software co-design paradigm.

By leveraging the Zynq-7000 SoC on the PYNQ-Z2 platform, the architecture effectively partitioned high-level dataset streaming to the ARM processor and latency-critical inference to a custom PicoRV32 RISC-V softcore. The rigorous engineering of a balanced, 3,503-sample dataset comprising thirteen distinct integrity metrics ensured the machine learning model was trained without majority-class bias. Crucially, the mathematical conversion of the Python-trained floating-point Support Vector Machine into a highly optimized, 32-bit fixed-point bare-metal C application proved that complex algorithmic security can be executed efficiently without a resource-heavy floating-point unit.

The experimental results unequivocally validate the proposed architecture. The hardware inference engine achieved a remarkable 99.4% classification accuracy against unseen data, perfectly distinguishing between normal, jammed, and spoofed signal states.

Furthermore, by bypassing standard operating system overhead, the bare-metal softcore demonstrated deterministic, zero-latency physical actuation via direct GPIO control. Ultimately, this thesis delivers a highly reliable, low-resource hardware accelerator capable of providing split-second threat intelligence, proving its viability for seamless integration into modern, edge-based navigation receivers.

6.2 Future Scope

While the current System-on-Chip prototype successfully demonstrates real-time inference and hardware actuation, there are several promising avenues for future research and architectural expansion. The primary focus for immediate development is physical live-sky RF integration. The current validation architecture relies on the ARM processor to stream pre-recorded, engineered GNSS datasets across the AXI BRAM. A critical next step is interfacing a physical, multi-constellation GNSS receiver directly to the FPGA fabric via UART or SPI, allowing the system to ingest and extract raw satellite data in real-time under genuine environmental conditions.

Another significant enhancement involves hardware-accelerated feature extraction. Currently, the thirteen feature metrics are pre-calculated prior to inference. Future iterations could move the feature engineering pipeline itself into dedicated hardware description language blocks on the programmable logic. By calculating pseudorange variance and signal-to-noise ratio gradients directly within the FPGA fabric, the design would evolve into a true, autonomous, end-to-end baseband security processor.

Addressing sophisticated multi-path spoofing attacks may also require transitioning to advanced non-linear machine learning models. While the fixed-point Linear SVM is exceptionally lightweight and highly accurate for the current dataset, highly complex RF interference might necessitate non-linear decision boundaries. Exploring the deployment of binarized neural networks or quantized deep neural networks on the FPGA could increase detection capabilities against advanced threats while actively maintaining a low computational footprint.

Finally, having proven the architecture on a reconfigurable Artix-7 FPGA, the design could ultimately be synthesized into an Application-Specific Integrated Circuit. An ASIC implementation would drastically reduce power consumption and the physical silicon footprint compared to a programmable logic device. This transition would make the embedded detection module highly suitable for widespread commercial deployment in micro-UAVs, autonomous vehicles, and smartphone navigation chipsets.

APPENDIX A: PROJECT PLAN AND WORK DIVISION

A.1 PERT Chart

The PERT (Program Evaluation and Review Technique) chart shown in Figure A.1 illustrates the sequential flow of tasks and their interdependencies in the GNSS spoofing and jamming detection project. It provides a clear visualization of critical activities and the project timeline, enabling the team to identify potential bottlenecks and prioritize tasks effectively. By mapping out each specific stage—from the parallel tasks of dataset collection and GNSS parameter study, through SVM training, to the final FPGA implementation and testing—the PERT chart ensured that the hardware-software co-design proceeded in a structured and time-efficient manner.

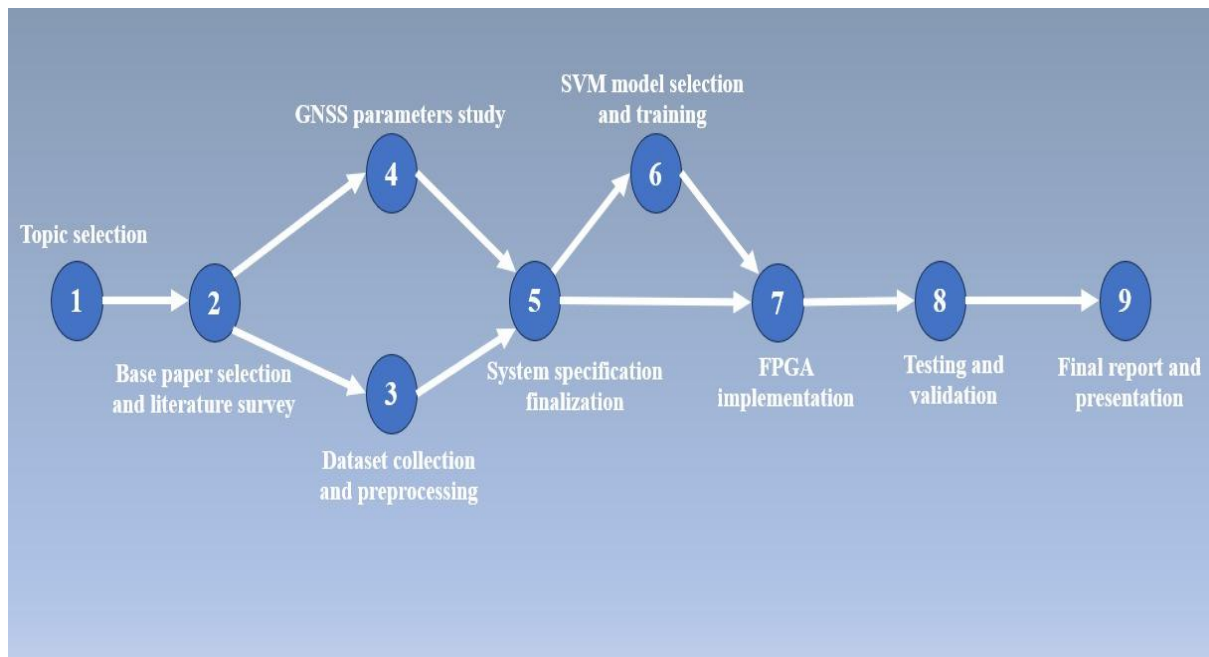


Figure A.1: PERT Chart

A.2 Gantt Chart

The Gantt chart shown in Figure A.2 outlines the project's complete timeline, spanning from initial topic selection in July 2025 to final documentation in March 2026. The initial phase

(July–October 2025) established the foundational machine-learning methodology, encompassing dataset preparation, feature extraction, and offline SVM training. The hardware realization phase commenced in December 2025, focusing on the RISC-V SoC bring-up, bare-metal C implementation of the inference engine, and physical integration on the PYNQ-Z2 platform. The project concluded in early 2026 with rigorous system-level testing and validation, illustrating a well-structured progression from software modeling to a successful hardware deployment.

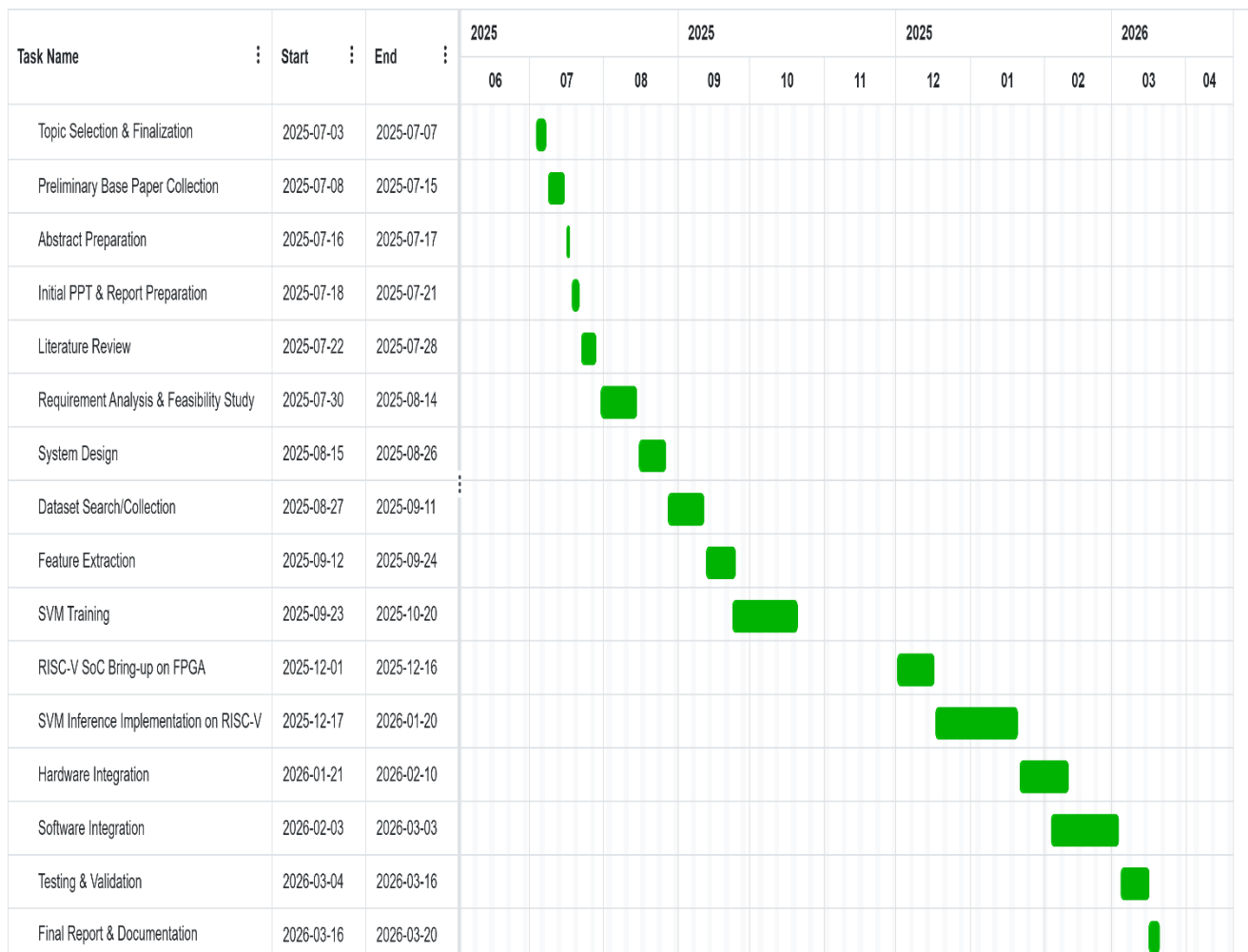


Figure A.2: Gantt Chart

A.3 Work Division

The successful realization of this hardware-software co-design was the result of a highly coordinated team effort. Responsibilities were strategically divided to cover all essential

domains of the project, spanning theoretical analysis, machine learning development, and physical hardware implementation. Key areas of focus during the software development stage included the comprehensive literature review, system architecture design, dataset engineering, and offline Support Vector Machine training.

Furthermore, the workload encompassed the critical hardware-centric tasks required for the final deployment. This included the Zynq-7000 SoC synthesis in Xilinx Vivado, bare-metal C firmware development for the PicoRV32 softcore, and real-time system validation on the PYNQ-Z2 platform. Each member also contributed to crucial project management tasks, including system-level debugging, final documentation, and technical presentations. The specific contributions and primary responsibilities of each team member are detailed in Table A.1.

Table A.1: Work Division

Member	Work Assigned
Nikhil Vinod	Dataset Pre-processing, Testing, Report, Presentation, Software Design, GNSS Integration, SVM Training
Pranav S	FPGA Interfacing, Hardware Software Integration, Output System, Hardware Design
Vaishakh Mohan	RISC V Setup, Presentation, Hardware Software Integration, Feasibility Study
Vishnu S Nambiar	Feature Extraction, RISC V Setup, GNSS Integration, SVM Training

APPENDIX B: SOURCE CODE

B.1 Feature Extraction Code:

```
import pandas as pd

try:
    # 1. Load the two source datasets
    gnss_data = pd.read_csv('merged_gnss_data.csv')
    # Use the new features file with pseudorange standard deviation
    rinex_features = pd.read_csv('advanced_rinex_features.csv')
    print("--- Step 1: Merging Datasets ---")
    # --- Prepare for the time-based merge ---
    gnss_data['real_time_x'] = pd.to_datetime(gnss_data['real_time_x'])
    rinex_features['time'] = pd.to_datetime(rinex_features['time'])
    gnss_data = gnss_data.sort_values('real_time_x')
    rinex_features = rinex_features.sort_values('time')

    # --- Perform the 'asof' merge ---
    final_dataset = pd.merge_asof(
        gnss_data,
        rinex_features,
        left_on='real_time_x',
        right_on='time',
        direction='nearest',
        tolerance=pd.Timedelta('1s')
    )

    # Remove any rows that didn't have a match
    final_dataset.dropna(subset=['avg_snr_L1'], inplace=True)
    print("Merge successful.")
```

```

print("\n--- Step 2: Selecting Features ---")

# 2. Define the list of columns to keep, including the new pseudorange feature
selected_features = [
    'agcCnt_01',      # Automatic Gain Control
    'jamInd_01',      # Jamming Indicator
    'noisePerMS_01',  # Noise per Millisecond
    'hMSL',           # Height above Mean Sea Level
    'velN',            # Velocity North
    'velE',            # Velocity East
    'velD',            # Velocity Down
    'hAcc',            # Horizontal Accuracy
    'vAcc',            # Vertical Accuracy
    'sAcc',            # Speed Accuracy
    'pDOP',            # Positional Dilution of Precision
    'avg_snr_L1',      # Average Signal-to-Noise Ratio
    'pseudorange_std_dev', # New Advanced Feature!
]

# 3. Create a new dataframe with only the selected features
features_df = final_dataset[selected_features]

# Rename the time column for clarity
features_df = features_df.rename(columns={'real_time_x': 'timestamp'})

# 4. Save the final, cleaned dataframe to a new file
output_filename = 'features_dataset_advanced.csv'
features_df.to_csv(output_filename, index=False)

print(f"Feature selection successful.")
print(f"\n✔ Success! Your final dataset is ready.")
print(f"Saved as '{output_filename}'")
print(f"\nThe new dataset has {features_df.shape[0]} rows and
{features_df.shape[1]} columns.")

```

```

    print("\nHere's a preview:")
    print(features_df.head())

except FileNotFoundError as e:
    print(f"Error: {e}")
except KeyError as e:
    print(f"KeyError: {e}. A specified column was not found.")

```

B.2 SVM Training Code:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.preprocessing import StandardScaler

# --- Configuration ---
SCALE_FACTOR = 8192 # 2^13
OUTPUT_FILE = "weights.h"
DATASET_FILE = 'dataset_balanced.csv'

# 1. Load Data
print(f"Loading {DATASET_FILE}...")
df = pd.read_csv(DATASET_FILE)
X = df.drop('label', axis=1)
y = df['label']

# --- PART A: Evaluation (For your Report) ---
print("\n--- Phase 1: Model Evaluation (Train/Test Split) ---")

```

```

# Split 80/20 just to calculate accuracy stats
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train on 80%
scaler_eval = StandardScaler()
X_train_scaled = scaler_eval.fit_transform(X_train)
X_test_scaled = scaler_eval.transform(X_test)

clf_eval = LinearSVC(dual=False, C=1.0, max_iter=10000)
clf_eval.fit(X_train_scaled, y_train)

# Evaluate
y_pred = clf_eval.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# --- TERMINAL OUTPUT FORMATTING ---
print("\n=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred, digits=2))

print("=== CONFUSION MATRIX ===")
print(cm)

print("\n=== MODEL SHAPES ===")
print(f"Classes      : {clf_eval.classes_.tolist()}")
print(f"Features      : {X_train.shape[1]}")
# Note: LinearSVC does not compute/store individual Support Vectors (SVs)
# like the standard SVC does. It computes the weights directly, which is
# why we use it for FPGA deployment!

print("\nModel and scaler parameters evaluated successfully.")

# --- PLOT CONFUSION MATRIX ---

```

```
plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues') # Changed to Blues to match
your previous style

plt.title(f'Confusion Matrix (Acc: {acc*100:.1f}%)')

plt.ylabel('True Label')

plt.xlabel('Predicted Label')

plt.show()
```

B.3 Bare-Metal Inference Engine Code:

```
#include <stdint.h>

#include "weights.h"

// --- MEMORY MAP (Using exact PicoRV32 addresses from your image) ---
#define INPUT_ADDR    ((volatile int32_t *)0x1000)
#define OUTPUT_ADDR   ((volatile int32_t *)0x2000)
#define RGB_LED       (*(volatile uint32_t *)0x40000000)
#define SEV_SEG       (*(volatile uint32_t *)0x40010000)

// --- SVM INFERENCE ENGINE ---
int32_t svm_predict_sev(int32_t *features) {
    int32_t best_class = -1;

    // Set baseline to the lowest possible 32-bit signed integer
    int32_t max_score = -2147483648;

    // 1. Iterate through each of the 3 classes (One-vs-Rest)
    for (int c = 0; c < NUM_CLASSES; c++) {
        // Use 64-bit integer to prevent overflow during multiplication
        int64_t score = 0;

        // 2. Calculate Dot Product (Weights * Features)
        for (int f = 0; f < NUM_FEATURES; f++) {
            score += (int64_t)SVM_WEIGHTS[c][f] * (int64_t)features[f];
        }
    }
}
```

```

    }

    // 3. Shift the decimal point back and add the Bias
    score = (score / SCALE_FACTOR) + SVM_BIAS[c];

    // 4. Argmax: If this class has the highest score so far, keep it
    if ((int32_t)score > max_score) {
        max_score = (int32_t)score;
        best_class = c;
    }
}

// Returns 0 (Normal), 1 (Jamming), or 2 (Spoofing)
return best_class;
}

int main() {
    int32_t last_result = -1;

    while (1) {
        int32_t result = svm_predict_sev((int32_t *)INPUT_ADDR);

        // Write to BRAM for Jupyter Console
        *OUTPUT_ADDR = result;

        if (result != last_result) {
            // 1. Update the RGB LED
            if (result == 0)      { RGB_LED = 0x2; } // Normal (Green)
            else if (result == 1) { RGB_LED = 0x4; } // Jamming (Red)
            else if (result == 2) { RGB_LED = 0x1; } // Spoofing (Blue)
            else                   { RGB_LED = 0x0; }

            last_result = result;
        }
    }
}

```

```

        // Short delay to prevent AXI bus saturation
        for(volatile int i=0; i<1500; i++);
    }
    return 0;
}

```

B.4 Real-Time Data Streaming and AXI Control (Jupyter Notebook):

```

from pynq import Overlay, MMIO
import numpy as np
import pandas as pd
import time

# --- Configuration ---
BITSTREAM_PATH = "system.bit" # Ensure this is your latest bitstream!
FIRMWARE_PATH  = "firmware.bin"
DATASET_PATH   = "dataset_balanced.csv"
BRAM_BASE      = 0x42000000
GPIO_RESET_BASE = 0x41200000

# NEW: The ARM address for the 7-Segment GPIO block
SEV_SEG_BASE   = 0x41220000

SCALE_FACTOR   = 8192

# Adjusted slightly so the 7-segment doesn't flicker too fast for human eyes
STREAM_DELAY   = 0.05

# --- SCALING VALUES ---
SCALER_MEAN = np.array([4057.8926634313448, 15.199330201857835,
134.9586032446246, 12.147823765812163, -0.010634125276620039, -
0.0001930981153297173, -0.0017126380536682845, 36.35678013282843,

```

```
0.4320764373739651, 0.06407865441735654, 0.010309407346560091, 783168.1907226901,
34179610.117281474])
```

```
SCALER_SCALE = np.array([2149.1019177110984, 9.354677767217579,
52.158371818110055, 2.373038659776659, 0.1357900990867289, 0.03629609883123221,
0.013159932160053548, 62.165559858709365, 0.18551853048633132,
0.016416749376823563, 0.000563101851462426, 1177599.3822929808,
43108223.284344725])
```

```
# 1. Setup Hardware
```

```
ol = Overlay(BITSTREAM_PATH)
```

```
bram = MMIO(BRAM_BASE, 0x10000)
```

```
reset_gpio = MMIO(GPIO_RESET_BASE, 0x1000)
```

```
sev_gpio = MMIO(SEV_SEG_BASE, 0x1000) # NEW: Map the 7-segment hardware
```

```
# NEW: Force GPIOs to be outputs to prevent floating bugs
```

```
reset_gpio.write(0x4, 0x0)
```

```
sev_gpio.write(0x4, 0x0)
```

```
# 2. Flash Firmware (PicoRV32 in Reset)
```

```
reset_gpio.write(0x0, 0)
```

```
with open(FIRMWARE_PATH, "rb") as f:
```

```
    words = np.frombuffer(f.read(), dtype=np.uint32)
```

```
    for i, word in enumerate(words):
```

```
        bram.write(i * 4, int(word))
```

```
# 3. Start PicoRV32
```

```
reset_gpio.write(0x0, 1)
```

```
print("PicoRV32 is running. Starting Live Stream...")
```

```
# 4. Load Dataset for Streaming
```

```
df = pd.read_csv(DATASET_PATH)
```

```
# NEW: Shuffle the dataset so you instantly see attacks!
```

```
df = df.sample(frac=1).reset_index(drop=True)
```

```
X_stream = df.drop('label', axis=1).values
```



```

y_stream = df['label'].values
class_names = {0: "NORMAL", 1: "JAMMING", 2: "SPOOFING"}

# 5. Continuous Streaming Loop
try:
    print(f"Streaming started at {STREAM_DELAY}s per sample. Press Kernel ->
Interrupt to stop.")
    while True:
        for i in range(len(X_stream)):
            # Normalize and Convert to Fixed Point
            raw_row = X_stream[i]
            scaled = (raw_row - SCALER_MEAN) / SCALER_SCALE
            fixed = (scaled * SCALE_FACTOR).astype(np.int32)

            # Inject Data into BRAM (Offset 0x1000)
            for j, val in enumerate(fixed):
                bram.write(0x1000 + (j * 4), int(val))

            # Delay for visual assessment
            time.sleep(STREAM_DELAY)

            # Read Prediction from BRAM (Offset 0x2000)
            prediction = bram.read(0x2000)

            # Python writes the prediction directly to your Verilog Decoder!
            sev_gpio.write(0x0, int(prediction))

            # Visual Feedback in Console
            print(f"Streaming Row {i:04d} | Predicted:
{class_names.get(prediction, 'UNKNOWN')} | True: {class_names[y_stream[i]]}
", end="\r")

except KeyboardInterrupt:
    print("\nStreaming stopped by user.")

```

APPENDIX C: PYNQ-Z2 BOARD LAYOUT AND PINOUT

The hardware implementation of the anomaly detection system heavily utilizes the integrated peripherals and general-purpose input/output (GPIO) headers of the PYNQ-Z2 development board. Figure C.1 illustrates the complete physical layout of the board, highlighting the locations of the onboard Zynq-7020 SoC, the power and boot jumpers, the standard communication ports (Ethernet, USB, HDMI), and the various expansion headers (Arduino, Raspberry Pi, and Pmods) available for external interfacing. For this project, the onboard User-Defined RGB LEDs and the external jumper headers were the primary components utilized for real-time visual threat actuation.

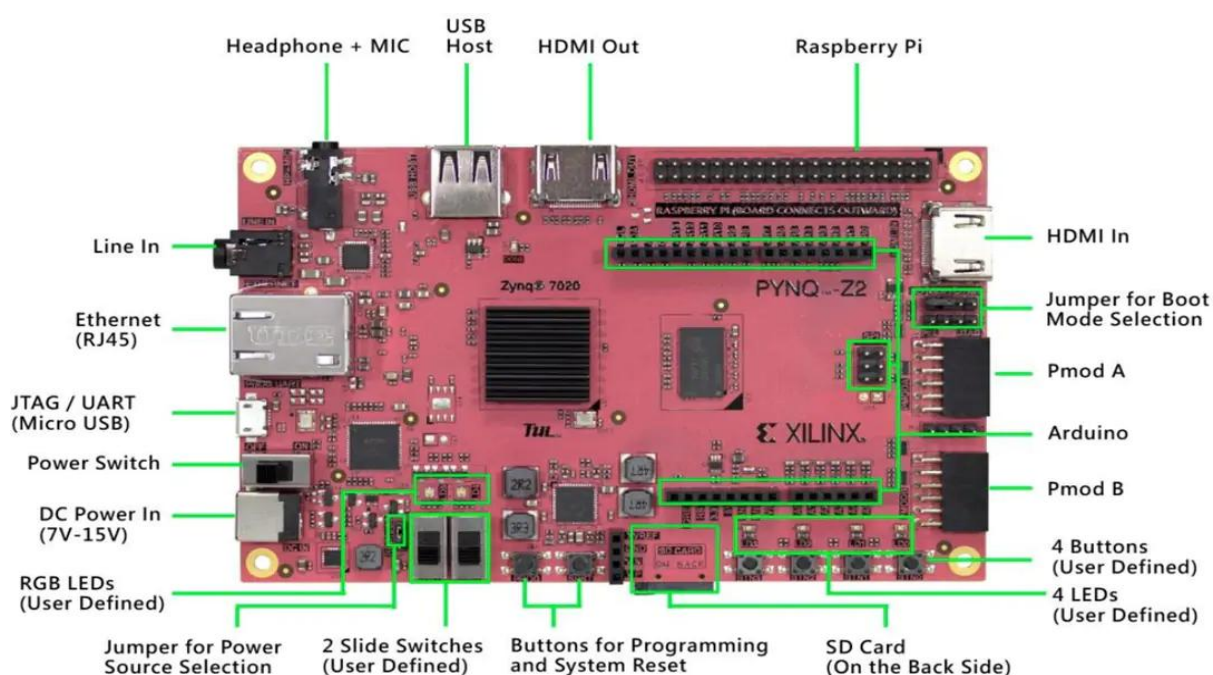


Figure C.1: Physical Layout & Peripheral Interfaces of the PYNQ-Z2 Development Board

While Figure C.1 displays the general location of the expansion headers, precise digital routing was required to bridge the RISC-V softcore and the ARM processor to the external 7-segment display. Figure C.2 provides the exact GPIO pinout mapping for these headers. This detailed schematic was strictly followed to ensure the external breadboard wiring aligned

perfectly with the AXI GPIO addresses synthesized in the Xilinx Vivado block design, preventing any electrical shorts or data routing errors during hardware execution.

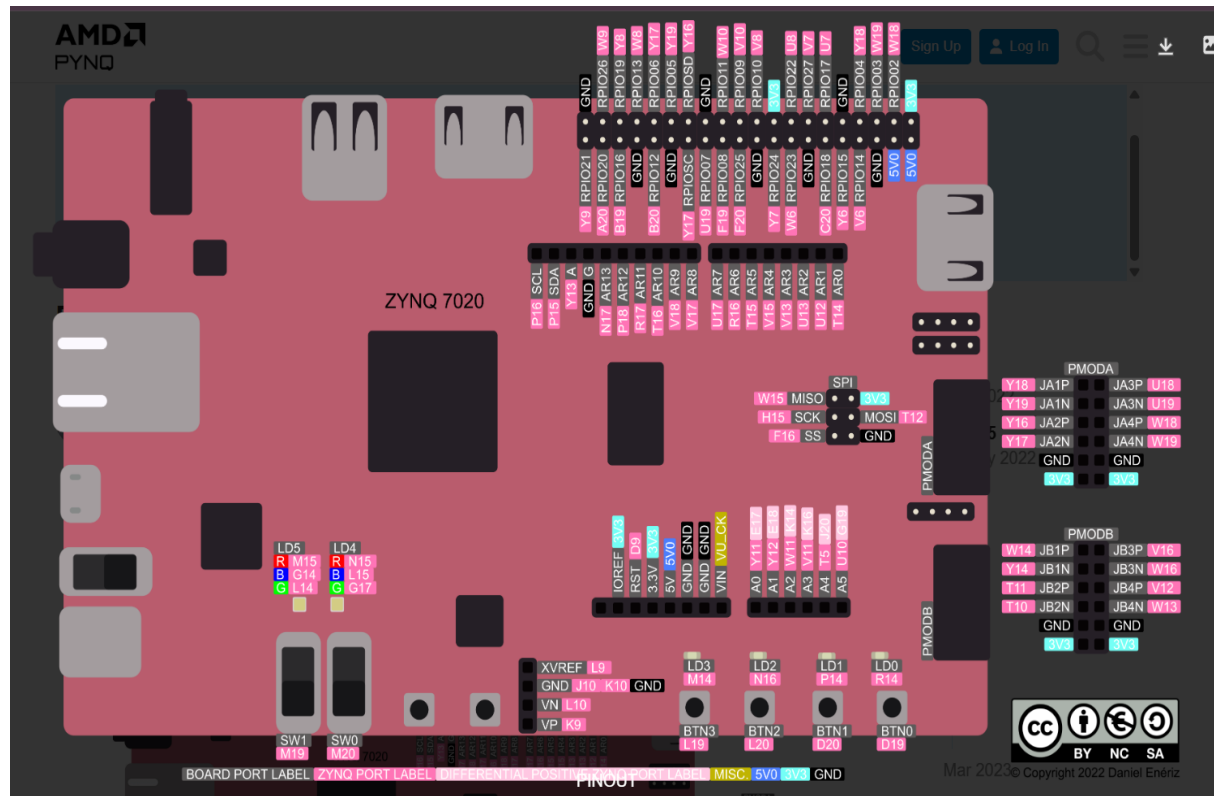


Figure C.2: GPIO Pinout Mapping of the PYNQ-Z2 FPGA Board

REFERENCES

- [1] A. Shafique, A. Mehmood and M. Elhadeif, "Detecting Signal Spoofing Attack in UAVs Using Machine Learning Models," in *IEEE Access*, vol. 9, pp. 93803-93815, 2021, doi: 10.1109/ACCESS.2021.3089847.
- [2] K. Radoš, M. Brkić and D. Begušić, "GNSS Signal Classification based on Machine Learning Methods," *2024 47th MIPRO ICT and Electronics Convention (MIPRO)*, Opatija, Croatia, 2024, pp. 806-811, doi: 10.1109/MIPRO60963.2024.10569174.
- [3] J. Li, X. Zhu, M. Ouyang, W. Li, Z. Chen and Q. Fu, "GNSS Spoofing Jamming Detection Based on Generative Adversarial Network," in *IEEE Sensors Journal*, vol. 21, no. 20, pp. 22823-22832, 15 Oct.15, 2021, doi: 10.1109/JSEN.2021.3105404.
- [4] Alberto Squatrito, "Machine Learning-based GPS Jamming and Spoofing Detection," *Doctoral Dissertations and Master's Theses*. 810., Embry-Riddle Aeronautical University, Daytona Beach, FL, USA, 2024.
- [5] Romaniuc, A.-G.; Vasile, V.-C.; Borda, M.-E., "Global Navigation Satellite System Spoofing Attack Detection Using Receiver Independent Exchange Format Data and Long Short-Term Memory Algorithm," *Information* 2025, 16, 502.
- [6] J. Li, Z. Chen, Z. Ran, Y. Xu, X. Zhu and X. Yuan, "The GNSS Spoofing Detection Method Based on AdaBoost," *2023 6th International Symposium on Autonomous Systems (ISAS)*, Nanjing, China, 2023, pp. 1-6, doi: 10.1109/ISAS59543.2023.10164411.
- [7] R. R. Yakkati, B. Pardhasaradhi, J. Zhou and L. R. Cenkeramaddi, "A Machine Learning based GNSS Signal Classification," *2022 IEEE International Symposium on Smart Electronic Systems (iSES)*, Warangal, India, 2022, pp. 532-535, doi: 10.1109/iSES54909.2022.00116.
- [8] A. Mahroof, I. Nabi, S. Z. Farooq and N. A. Naqvi, "Machine Learning-Based Detection of Spoofing Attacks in GNSS: A Study Using TEXBAT Dataset," *2024 14th International Conference on Electrical Engineering (ICEENG)*, Cairo, Egypt, 2024, pp. 90-95, doi: 10.1109/ICEENG58856.2024.10566287.

- [9] M. S. Meraz, J. M. C. Arvizu and A. J. A. Cruz, "GNSS Receiver Based on a SDR Architecture Using FPGA Devices," *2011 IEEE Electronics, Robotics and Automotive Mechanics Conference*, Cuernavaca, Mexico, 2011, pp. 383-388, doi: 10.1109/CERMA.2011.70.
- [10] Ghanbarzade, A. and Soleimani, H., "GNSS/GPS Spoofing and Jamming Identification Using Machine Learning and Deep Learning", <i>arXiv e-prints</i>, Art. no. arXiv:2501.02352, 2025. doi:10.48550/arXiv.2501.02352.
- [11] A. Noah and M. Elmusrati, "GNSS Spoofing and Jamming Mitigation: A Comprehensive Review," 2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA), Antalya, Turkiye, 2025, pp. 1-9, doi: 10.1109/ACDSA65407.2025.11165996.
- [12] B. Kandel and N. Acharya, "Characterizing GNSS Signal Behavior Under Jamming and Spoofing using Doppler, Pseudorange, and C/N0 Metrics," 2025 International Conference on Information Science and Communications Technologies (ICISCT), Kathmandu, Nepal, 2025, pp. 156-160, doi: 10.1109/ICISCT68600.2025.11441494.
- [13] P. Kapadia, A. A. Chitale, K. Kushwaha, S. Dandale, S. Jadhav and M. Pagale, "GPS Spoofing and Jamming in Drones: A Growing Threat to UAV Security," 2025 9th International Conference on Computing, Communication, Control and Automation (ICCCBEA), Pune, India, 2025, pp. 1-5, doi: 10.1109/ICCUBEA65967.2025.11284125.
- [14] Y. Pervysheva, J. Käppi, J. Syrjärinne, J. Nurmi and E. S. Lohan, "Multi-Receiver Ensemble Machine-Learning Techniques for GNSS Spoofing Detection with Generalisability Approach," 2024 32nd Telecommunications Forum (TELFOR), Belgrade, Serbia, 2024, pp. 1-4, doi: 10.1109/TELFOR63250.2024.10819159.
- [15] N. Z. B. Taufik, G. E. M. Abro, S. Hafeez, M. A. Zulkifley and N. M. Tahir, "Simulation-Driven Analysis of GPS Jamming and Spoofing Effects on UAV Flight Stability," 2025 IEEE 15th International Conference on System Engineering and Technology (ICSET), Kuala Lumpur, Malaysia, 2025, pp. 429-433, doi: 10.1109/ICSET65917.2025.11283654.
- [16] B. Singh and N. Gupta, "Jamming Detection of Global Navigational Satellite System utilizing Deep Learning," 2025 IEEE 3rd Global Conference on Wireless Computing and Networking (GCWCN), Lonawala, Maharashtra, India, 2025, pp. 1-5, doi: 10.1109/GCWCN66157.2025.11448371.

- [17] S. Bertram, L. Eisentraut and R. Buettner, "A Systematic Literature Review of Current Machine Learning Approaches for Detecting GNSS Spoofing Attacks," in IEEE Access, vol. 13, pp. 108898-108917, 2025, doi: 10.1109/ACCESS.2025.3582435.
- [18] V. S. Kumar, G. N. Mori, S. K. K. R, P. J. Josephson, N. Kumar and R. Nithya, "Detecting Jamming and Spoofing Attacks on Unmanned Aerial Vehicles with Advanced Neural Network Models," 2025 Global Conference in Emerging Technology (GINOTECH), PUNE, India, 2025, pp. 1-6, doi: 10.1109/GINOTECH63460.2025.11076803.
- [19] Y. Pervysheva, J. Käppi, J. Nurmi and E. S. Lohan, "GNSS interference detection and classification with real-field Jammertest 2024 data and neural networks," Research Square, preprint, Nov. 9, 2025. doi: 10.21203/rs.3.rs-7864272/v1.
- [20] Broumandan, A.; Pirsiavash, A.; Tremblay, I.; Kennedy, S. Jamming and Spoofing Detection and Classification Performance Under Hostile GNSS Environments. Eng. Proc. 2025, 88, 76. <https://doi.org/10.3390/engproc2025088076>.
- [21] Xu, D., Li, H., Wang, Y., Huang, D., Zhao, H., Chi, J., & Wu, Q. (2026). The jamming and spoofing of GNSS in Persian Gulf during the Israel-Iran conflict: an analysis by receiver data. Advances in Space Research.
- [22] Y. Zeng et al., "GNSS Jamming and Spoofing Threats in UAV Navigation: Countermeasure Status and Challenges," in IEEE Communications Surveys & Tutorials, vol. 28, pp. 5909-5948, 2026, doi: 10.1109/COMST.2026.3680438.
- [23] M. I. Sayyaf, Miguel Ortiz and V. Renaudin, "GNSS Dataset Under Jamming, Spoofing, and Meaconing Conditions". Zenodo, Jul. 15, 2025. doi: 10.5281/zenodo.15911589.